

密级：公开

北京邮电大学

硕士学位论文



题目：基于API序列的深度学习恶意软件检测方法
的研究与实现

学 号：2021141004
姓 名：刘世一
学科专业：计算机技术
培养方式：全日制
导 师：苑洁
学 院：网络空间安全学院

2024 年 06 月 04 日

中国·北京

密级：公开

北京邮电大学

硕士学位论文(专业学位)



题目： 基于 API 序列的深度学习恶意软件检测
方法的研究与实现

学 号： 2021141004

姓 名： 刘世一

学科专业： 计算机技术

培养方式： 全日制

导 师： 苑洁

学 院： 网络空间安全学院

2024 年 6 月 4 日



BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

Master Dissertation

Research and Implementation of Deep Learning
Malware Detection Method Based on API Sequence

Student ID: 2021141004

Author: LiuShiYi

Subject: Computer Technology

Supervisor: YuanJie

Institute: School of Cyberspace Security

Date 4 Month 6 Year 2024

答辩委员会名单

职务	姓 名	职 称	工 作 单 位
主席	陆月明	教授	北京邮电大学
委员	王伟	教授	北京交通大学
委员	姚文斌	教授	北京邮电大学
委员	关振宇	教授	北京航空航天大学
委员	于光明	副研究员	中国通用技术研究院
秘书	蔡斌思	讲师	北京邮电大学
答辩日期	2024 年 5 月 22 日		

摘 要

随着计算机技术的发展和互联网的广泛应用,网络已经深入到生活的方方面面,但网络威胁也随之而来,恶意软件就是主要威胁之一。随着自动化、人工智能等技术的发展恶意软件不断进化,新型恶意软件的产生速度越来越快,隐藏特征躲避检测的手段越来越复杂。而恶意软件检测模型依赖人工更新,速度慢,检测变得困难。因此,恶意软件检测技术研究变得尤为重要。

本文对恶意软件检测相关技术、国内外研究现状进行调研,对当前恶意软件检测技术发展进行了充分了解。针对 Windows 系统上的可执行文件,当前有些恶意软件检测模型在准确率、鲁棒性上表现不理想和模型更新速度慢的问题,提出了使用 API 序列的深度学习恶意软件检测模型,完成了恶意软件检测系统设计与实现,主要工作如下:

(1) 提出了基于 API 调用序列的深度学习恶意软件检测模型 Mal-CLAM,该模型结合 API 序列结构特征和上下文依赖关系进行检测。本文使用的 API 序列包含名称、类别、使用参数等信息,相比其它使用单一特征的 API 序列包含的内容更加丰富。Mal-CLAM 模型使用自编码器获取低维深度特征并重建数据获取重构误差作为结构特征;使用门控卷积和 Bi-LSTM 来捕获 API 序列间的依赖关系;最后将特征融合使用 MLP 模型进行恶意软件检测。模型的特征收集处理无需人工参与,减少了模型更新消耗的时间。进行了消融实验探究每个模块起到的作用和对模型的影响。与现有方法相比, Mal-CLAM 模型达到了 94.49%的准确率、92.75%的召回率、93.63%的 F1 值和 98.43%的 AUC 值,总体表现更佳,鲁棒性更强。

(2) 设计并实现了基于 Mal-CLAM 模型的使用 API 序列的深度学习恶意软件检测系统,面向用户提供恶意软件检测及相关服务,包括登录注册、恶意软件检测、模型更新、历史记录搜索、可视化等功能。恶意软件检测系统接收用户上传的软件样本进行检测,并将软件特征和检测结果存储,利用上传检测的软件更新模型,以应对不断更新迭代的恶意软件攻击;此外,系统提供了搜索功能可以根据软件信息查询对应检测结果,提供了可视化功能展示近期的检测记录。本文对恶意软件检测系统进行了较为全面的测试,证明了系统的可用性。

关键词: 恶意软件检测; 深度学习; API 序列; 自编码器; 神经网络

ABSTRACT

With the development of computer technology and the widespread use of the Internet, networks have permeated every aspect of life. However, along with this expansion comes network threats, with malicious software being a primary concern. As technologies like automation and artificial intelligence advance, malicious software continues to evolve at an increasing pace, with new variants emerging rapidly and employing increasingly complex methods to conceal their characteristics and evade detection. Meanwhile, detection models for malicious software rely on manual updates, which are slow, making detection increasingly challenging. Therefore, research into malicious software detection technology has become particularly important.

In this thesis, we investigate the related technology of malware detection, the current research status at home and abroad, to fully understand the current status of malware detection technology. On Windows systems, aiming at the problems that some current malware detection models do not perform satisfactorily in terms of accuracy and robustness and the slow update speed of the model, this thesis proposes a malware detection model based on deep learning and carries out the design and realization of the malware detection system, and the main work is as follows:

(1) Proposed Mal-CLAM model, a deep learning malware detection model based on API call sequences, which combines structural features of API sequences and context-dependent model detection. The API sequences used in this thesis contain information such as name, category, and parameters used, etc. In the implementation, the Mal-CLAM model uses an autoencoder to obtain low-dimensional depth features and reconstructs the data to obtain reconstruction errors as structural features; gated convolution and Bi-LSTM are used to capture the dependencies between API sequences; and finally, the two types of features are fused to be detected using the MLP model to enhance the effectiveness of malware detection. The feature collection process used in this thesis requires no human involvement and reduces the time consumed for model update.

Ablation experiments are conducted to explore the role of each module on detection, and the impact of each module on the model is analyzed to provide a basis for optimizing the structure of the detection model. Compared with existing methods, the Mal-CLAM model achieved 94.49% accuracy, 92.75% recall, 93.63% F1 value and 98.43% AUC value (area under the curve), which is a better overall performance and robust.

(2) Designed and implemented a malware detection system based on the Mal-CLAM model to provide malware detection and related services for users, including login and registration module, malware detection module, model update module, history search module, and visualization module. The malware detection system can receive software samples uploaded by users for detection, store software features and detection results, and update the model according to the software uploaded and detected by users to cope with the constantly updated and iterative malware attacks; in addition, the system provides a search function to query the corresponding detection results according to the software information, and provides a visualization function to display the recent detection records. In addition, this thesis provides a more comprehensive test of the system to demonstrate its usability.

KEY WORDS: Malware Detection; Deep Learning; API Sequences; Autoencoder; Neural Networks

目 录

第一章 绪论.....	1
1.1 研究背景与意义.....	1
1.2 国内外研究现状.....	2
1.2.1 传统恶意软件检测技术.....	3
1.2.2 基于深度学习的恶意软件检测技术.....	5
1.3 研究内容.....	8
1.4 论文组织架构.....	9
第二章 恶意软件检测及相关技术介绍	11
2.1 恶意软件及对抗技术.....	11
2.1.1 恶意软件定义及类别.....	11
2.1.2 恶意软件对抗技术.....	12
2.2 API 调用序列	13
2.3 恶意软件检测技术.....	15
2.3.1 卷积神经网络.....	15
2.3.2 长短时记忆网络.....	16
2.3.3 自编码器.....	18
第三章 基于 API 序列的深度学习恶意软件检测模型.....	19
3.1 问题分析和模型框架.....	19
3.1.1 恶意软件检测技术问题分析.....	19
3.1.2 恶意软件检测模型框架.....	20
3.2 基于 API 序列结构特征和依赖关系的恶意软件检测模型.....	21
3.2.1 Mal-CLAM 检测模型架构	21
3.2.2 基于门控卷网络的特征选择.....	22
3.2.3 基于 Bi-LSTM 的依赖关系提取.....	23
3.2.4 基于自编码器的结构特征提取.....	24
3.2.5 基于多层感知器的恶意软件检测.....	26
3.3 实验设置.....	27
3.3.1 实验环境.....	27
3.3.2 实验数据.....	27
3.3.3 评价指标.....	29
3.4 消融实验.....	30
3.4.1 门控卷积消融实验.....	30
3.4.2 Bi-LSTM 消融实验	31
3.4.3 自编码器消融实验.....	32

3.5 实验结果及分析.....	32
3.5.1 恶意软件检测模型性能评估.....	32
3.5.2 恶意软件检测模型抗干扰能力评估.....	34
3.6 本章小结.....	36
第四章 基于深度学习的恶意软件检测系统设计与实现	37
4.1 系统需求分析与设计.....	37
4.1.1 系统需求分析.....	37
4.1.2 系统架构设计.....	38
4.1.3 系统模块设计.....	39
4.2 恶意软件检测系统实现.....	40
4.2.1 登录注册模块实现.....	41
4.2.2 恶意软件检测模块实现.....	41
4.2.3 模型更新模块实现.....	43
4.2.4 历史记录搜索模块实现.....	43
4.2.5 可视化模块实现.....	44
4.3 系统测试.....	45
4.4 本章小结.....	51
第五章 总结与展望	53
5.1 研究工作总结.....	53
5.2 未来工作展望.....	54
参考文献.....	55

第一章 绪论

随着科技的不断发展，网络已经深入渗透到我们生活的方方面面，极大地便利了我们的生活。与之相对的，越来越多的不法分子利用恶意软件进行攻击，破坏网络空间环境，这不仅使个人利益受到侵犯，甚至是企业、国家也遭受到了巨大的损失。因此研究恶意软件检测技术，维护网络空间安全是一项艰巨而又意义重大的工作。

1.1 研究背景与意义

随着计算机技术的飞速发展，计算机软件已经深刻渗透到我们生产和生活的方方面面。这使得恶意软件的威胁变得更加严峻，因为攻击者不仅能够侵入个人用户的系统，还能够对企业、政府和其他机构造成严重的影响^[1]。因此，应对不断演变的网络威胁，确保操作系统的安全性和用户隐私的保护变得至关重要。浏览记录、网上发言、出行记录、企业员工信息等大规模、高质量、敏感性的信息数据源源不断地流入计算机软件系统，更是使得恶意软件攻击的收益越来越大，这也吸引了越来越多不法分子发动网络攻击。伴随着各类新技术的不断创新和发展，新型恶意软件的出现速度和破坏力与日俱增。

根据 2023 年 AV-TEST 的最新数据，截至 2023 年底，新增恶意软件数量已超过 8000 万。图 1-1 是对近些年来恶意软件产生数量的统计^[2]，可以看到自 2015 年以来，每年新恶意软件的产生量一直稳定超过 8000 万，由此导致的经济损失不可估量。预计到 2025 年，因恶意软件造成的经济损失将达到十万亿美元。近些年来来的重大网络攻击事件，如 WannaCry 勒索软件、SolarWinds 供应链攻击和 Colonial Pipeline 勒索软件攻击，就是当今网络空间安全形势的缩影。

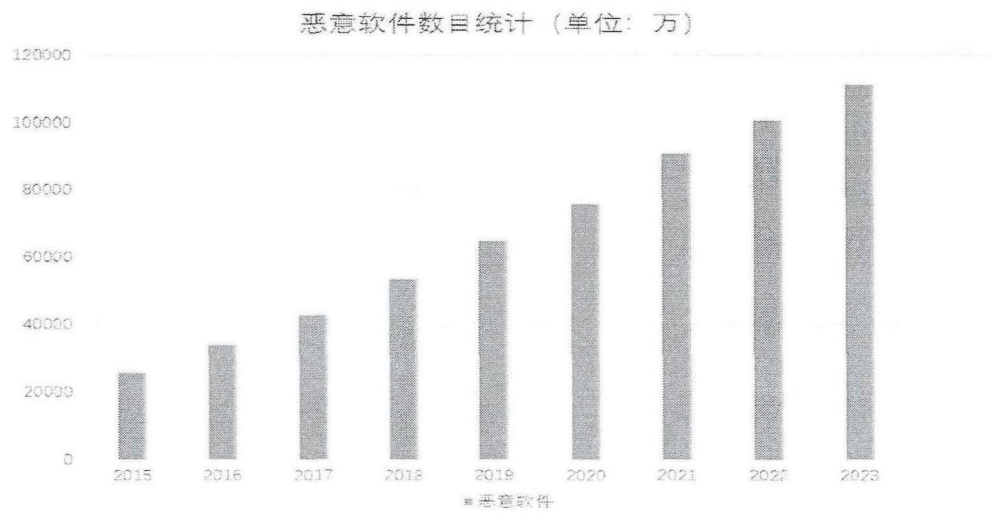


图 1-1 恶意软件数量统计

WannaCry 勒索软件攻击爆发于 2017 年 5 月,对全球范围内的计算机系统造成严重影响^[3]。WannaCry 利用操作系统的漏洞实施攻击,感染了数十万台计算机。一旦感染,它便立即夺取用户文件的控制权限,进行加密操作让用户无法访问自己的文件,并要求受害者支付赎金,否则便将文件销毁。许多用户出于工作安全的考虑纷纷支付赎金。此次攻击引起了全球关于网络安全和数字基础设施脆弱性的广泛关注,它强调了对操作系统及时更新、漏洞修复的必要性,以及对网络安全防护措施的重视。

SolarWinds 供应链攻击是发生在 2020 年的一场大规模网络攻击事件^[5],攻击者通过篡改 SolarWinds Orion 的软件更新包,将恶意代码植入软件中,然后通过合法的软件更新通道将恶意的更新分发给数万用户。这种攻击方式使得被攻击的组织在未察觉的情况下安装了恶意软件,成功了渗透数百家政府机构、企业 and 非营利组织。这次攻击引发了对全球供应链上网安全的广泛担忧,并促使了对软件供应链和网络安全的加强和审视。

Colonial Pipeline 勒索软件攻击发生于 2021 年 5 月,是一起影响美国能源供应链的严重网络攻击事件^[6]。攻击导致石油管道被迫关闭,最终造成油价波动、能源紧缺,甚至引发恐慌性购买。Colonial Pipeline 公司最终不得不支付约 450 万美元的比特币赎金。该事件引起对世界各国政府对关键基础设施网络安全的关切,强调了网络攻击对现代社会的影响。

2023 年 10 月,德国地方市政服务提供商 Südwestfalen IT 公司的服务器被未知的黑客团伙加密^[4],公司被限制了 70 多个城市对基础设施的访问权限,官方网站已经无法访问,这次攻击令地方政府服务“严重受限”。

更为严峻的是,恶意软件反检测技术^[7]也在不断改进,攻击者不断地使恶意软件更新变异,产生更多变体。如 Sodinokibi 勒索病毒^[8]早期能够利用 Web 服务的漏洞进行传播,后来经过攻击者的升级改造,可以伪装成 PDF 文件、Excel 文档、Word 文档等格式通过垃圾邮件进行传播^[9]。随着技术的发展,恶意软件更新速度越来越快,而恶意软件检测模型的更新需要领域专家的参与,导致模型的更新存在较长的时间延迟,难以满足对新型恶意软件及变体的及时检测要求^[10]。

高性能检测和快速更新的恶意软件检测技术能够对网络上的恶意软件进行检测,及时发现新型、隐蔽、高风向的恶意软件,阻止其入侵计算机系统开展恶意攻击行为,这项技术在维护网络安全、保护用户信息和财产以及维护社会稳定方面发挥着至关重要的作用,具有不可或缺的重要性。

1.2 国内外研究现状

由于恶意软件的肆虐,给用户、企业甚至国家都造成了巨大的危害,国内外

许多信息安全的研究人员致力于研究恶意软件检测技术,从风险发生根源进行阻断。到现在已经有很多恶意软件检测技术受到了广泛地应用。

1.2.1 传统恶意软件检测技术

目前,主流的恶意软件检测方案包括静态分析、动态分析以及动静结合分析方法^[11]。静态分析技术检测速度快、准确率高,但只能针对当前已被研究的恶意软件,难以识别新型恶意软件和通过混淆、加壳的软件;动态分析技术检测能力更强,可以有效检测出经过伪装的恶意软件,但是需要在安全虚拟环境中运行软件,资源消耗大、响应时间相对长;动静结合分析技术结合上述两种方法,静态分析可以在不执行程序的情况下检测代码中的潜在问题,而动态分析则可以模拟真实运行环境中的行为,两者相结合能够提高分析的效率,更全面地检测和分析软件系统中的问题,但是其模型更加复杂、维护更加困难、实施成本高昂。

(1) 静态分析技术

静态分析并不执行软件,而是通过反编译软件样本根据其而底层代码进行词法分析、语法分析、语义分析、控制流分析等进行检测。一般需要通过反汇编软件(如针对移动主机的 IDA、针对安卓系统的 Apktool),获取软件的字节码、操作码、源代码等特征进行检测^[12]。常被用于静态分析的特征有:操作码序列、DLL 调用函数、PE 头文件等。基于静态分析的恶意软件检测技术主要包括基于签名的检测技术、基于启发式的检测技术等。

基于签名的检测技术^[13]是一种传统的恶意软件检测方法,依赖于已知恶意软件的特定标识(签名)来辨别潜在威胁。这种检测方法主要通过比对文件或程序的二进制数据与已知的病毒或恶意软件签名数据库中存储的数据进行匹配,达到快速检测网络上已知的恶意软件并向用户提出警报的目的。尽管基于签名的检测技术能够高效地识别已经被记录研究的威胁,但它存在无法有效对抗新型、未知恶意软件的局限性。

随着技术的飞速发展,恶意软件攻击日趋复杂和隐蔽,使得传统基于签名的检测方法在应对新兴威胁方面显得力不从心。这是因为新型恶意软件可能采用多种变异和伪装手段,改变了原有特征的表现形式,以规避传统检测机制,所以面对未知、创新型威胁时,它的应变能力相对较弱,给系统留下一定的安全漏洞。尽管如此,基于签名的检测仍然是网络安全策略中重要的组成部分,特别是用于阻止已知的、常见的威胁,也仍有许多研究人员在该方向上不断探索。F. Zolkipli 等人^[14]利用遗传算法,使用签名生成器、静态分析检测器和遗传算法检测器构成检测模型,三个主要组成部分将作为相互关联的过程一起工作,完成了自动生成签名并检测的完整流程;Borojerdi 等人^[15]提出了一种基于序列聚类的 MalHunter 检测系统,系统运行时会分析收集软件的行为序列并与之前存储在样本数据库的

行为序列组进行比较以判断软件的性质。

基于启发式的恶意软件检测方法利用经验法则和直觉而不是明确的规则的方法来识别潜在的恶意软件^[16]。启发式方法可以检测未知的恶意行为，因为它们关注软件的行为和模式而非具体的签名，这种检测方法能够灵活适应新型威胁，但产生误报的概率也相对较大，并且无法检测到复杂的恶意软件。Yanfang 等人提出了一种用于恶意软件检测的关联分类的后处理技术^[17]，通过使用基于卡方检验规则排序机制的数据库覆盖率和悲观误差估计的规则修剪、规则排序和规则选择方法大大减少了生成规则的数量，然后通过选择最佳的规则进行预测；Arnold 等人中提出了一个自动生成的启发式恶意软件检测模型^[18]，该模型构建多个神经网络分类器，通过使用投票程序组合单个分类器输出进行检测。

恶意软件静态分析技术一般需要该领域的专家手动分析、提炼恶意软件的特征，将之更新到恶意软件数据样本库中，需要大量的人工参与。这些检测方法虽然检测速度和准确很高，但这种效果只能针对已经出现，并被专业人员研究解析的恶意软件。当前基于静态分析的恶意软件检测技术难以面对代码混淆加壳^[19]、“零日”恶意软件^[20]等攻击。使用代码混淆、加壳处理过的恶意软件会导致主流恶意软件的检测准确率大幅下降，攻击者可以利用代码混淆技术大大降低现有方法的有效性，并绕过许多恶意软件检测系统；“零日”恶意软件是指从未见过的恶意软件，或者是新出现的、缺乏有效知识的恶意软件，这种新颖性和现有缓解策略的缺乏使得零日恶意软件难以被检测和防御^[21]。

（2）动态分析技术

动态分析是通过在安全的虚拟环境中运行程序并捕获程序的重要行为特征来进行检测的^[22]，如 API 序列，这种方法不依赖于先验知识，而是关注实际的执行行为。将软件样本放置到安全虚拟环境^[23]中运行，并记录其在运行过程中执行的各种操作、调用的 API 函数。这包括文件操作、网络通信、进程创建、注册表修改等活动，通过观察和分析这些行为，安全专家可以确定程序是否表现出典型的恶意特征。虽然恶意软件可以采取各种隐蔽手段，但最终还是会执行必要的恶意操作来获取敏感信息，因此动态检测相对来说更加准确和稳健。

LuXiaoFeng 等人^[24]使用动态分析的方法，通过分析 API 调用序列中的依赖关系和冗余信息进而推测当前 API 调用的语义来进行恶意软件检测。该方法将深度学习和机器学习模型结合用于恶意软件行为分析，其中一部分从功能层面分析使用随机森林学习 API 序列特征；另一部分将残差添加到双向 LSTM 模型中，利用其可以提取序列前一个上下文和后一个上下文之间的依赖关系的特性训练更深层次的神经网络提高恶意软件检测性能。

Das 等人^[25]聚焦于捕获恶意软件的恶意行为，即高级语义，提出了一种以频

率为中心的恶意软件检测模型,利用恶意和良性样本不同的系统调用模式分别构建特征,输入到 MLP 中训练。该模型在软件运行时利用获取的不完全信息即时生成特征进行早期预测。该方法可以在不完全将软件运行完毕的情况下进行早期地恶意软件检测,在执行的前 30%时间内检测到了 46%的恶意软件,在执行 100%时检测到了 97%的恶意样本。

Cai 等人^[26]结合基于静态分析的 MamaDroid 模型,提出了基于动态分析的 DroidSpan 模型,研究了恶意软件检测器的可持续性问题。该方法基于应用程序行为配置文件,从轻量级分析中捕获敏感的访问分布情况,通过定义可持续性检测指标优化模型的检测性能,是检测模型能够随着时间的推移维持其功能,而无需频繁的重新训练。通过与五种其它的恶意软件检测器进行比较,表明了 DroidSpan 在准确性和持续性方面的优越性。

动态分析技术的挑战在于模型训练运行资源消耗大、成本高,以及如何处理大量的数据,生成准确的行为模型,并迅速判断是否存在恶意行为。

(3) 动静结合的分析技术

动静结合的恶意软件检测技术^[27],也称混合分析技术,利用了两者的优势,可以提高检测的准确性和效率。静态分析可以在不运行程序的情况下快速识别潜在的恶意特征,而动态分析则可以捕获程序实际执行时的行为,从而更准确地确定是否存在恶意活动。这种技术的优点包括对抗恶意软件变种的能力更强,能够识别出新型威胁,并且在运行时不会对系统造成影响。

在混合分析中,静态特征通常选取权限许可、操作码等;动态特征通常选取 API 系统调用、CPU 信息、IP 地址等。Andrea 等人^[28]提出一种动静结合的恶意软件检测系统 MADAM,它同时分析软件内核、应用程序、用户和调用包四个层次的特征进行检测;Wen 等人^[29]以权限、意图、使用特征、应用程序作为静态特征,以 CPU 消耗、电池消耗、运行进程的数量和短消息的数量作为动态特征。然后使用主成分分析和特征权重算法来降低特征的维数,使用支持向量机构建模型进行恶意软件检测;Jian 等人^[30]提出了一种基于融合特征的恶意软件检测系统,它使用 377 个动静混合特征,设置十个类别,采用属性子集选择和主成分分析来降低融合特征的维数,使用随机森林进行检测,有效提高了检测的准确性。

动静结合的分析技术存在静态分析的信息不完整和动态分析的高资源消耗等问题,而且模型结构更加复杂、调整优化难度大,处理不好反而会导致结果适得其反。

1.2.2 基于深度学习的恶意软件检测技术

近年来,深度学习技术(Deep Learning, DL)被广泛应用于各种领域,如医学影像识别、生物特征识别、智能问答、音频分析等^[31]。深度学习通过一个较为

复杂的网络结构学习数据的深层特征^[32],其最终目标是使计算机可以像人那样拥有语言分析的能力,并且可以分辨文本、图形和音频方面的内容^[33]。深度学习利用结构复杂的算法模型,在语言和图像识别方上取得了非常巨大的突破,很多研究成果已经被运用于实际生活中,带来了极大的便利^[34]。近年来,深度学习技术已被广泛应用于对恶意软件的研究^[35]。各种类型的深度学习算法,例如卷积神经网络^[36]、循环神经网络^[37]和前馈神经网络^[38],都已经被广泛应用在使用字节序列、灰度图、结构熵、以及 API 的恶意软件分析系统中的各种用例调用顺序,并取得了优异的效果。基于深度学习的恶意软件检测技术,可以有效解决特征泛化能力差,分类能力不足等问题。

Naeem 等人^[39]将视觉神经网络引入到物联网环境中的恶意软件检测,提出了恶意软件图像分类系统 MICS,该系统将恶意软件文件转换为灰度图像,通过 D-SIFT 特征描述符将图像分为众多补丁并从中提取局部特征,通过 GIST 特征描述符对图像进行滤波处理,提取全局特征,最后通过特征集成获取混合特征,并训练 SVM 分类器,在恶意软件家族分类上取得了不错的效果。Xing 等人^[40]提出一种基于自编码器重构误差检测模型,构建恶意软件的灰度图像,通过自编码器提取恶意软件的低维特征实现恶意软件检测。

Hassen 等人^[41]提出了一种基于控制语句的静态分析恶意软件分类技术,该方法从反汇编的恶意二进制文件中提取特征,使用基于控制语句叠加方法处理特征,最后使用随机森林算法进行分类,检测结果良好。但该方法泛化能力差,恶意软件行为稍加改变,检测效果就会下降。

Alzaylaee 等人^[42]提出了采用深度神经网络算法的 DL-Droid 模型,该模型从软件样本中动态提取 178 个特征,即 API 调用和意图-操作/事件,使用信息增益对特征进行排序,然后选择前 120 个特征使用深度神经网络进行检测,检测效果良好。但该方法需静态提取软件权限,在运行时动态提取 API 调用和意图,资源消耗大,算法模型结构复杂。Raff 等人^[43]使用深度卷积神经网络对整个软件样本的字节码文件进行分析,解决了时间步数超过 200 万的序列问题,检测准确率达到 94%,但是该算法完全运行训练一次需要的将近 200 小时,时间和资源成本很大。

目前,API 序列被广泛应用于基于动态分析的恶意软件检测中,目前已经取得了许多具有实际使用价值的成果。

Lei 等人^[44]提出了一种 API2Vec 的方法,使用时态进程图和时态 API 图捕获软件恶意行为进行检测,但是该算法启发式随机游走算法获取 API 执行路径,导致获取的执行路径重复,序列长度不一,影响模型准确率;Zou 等人^[45]将 API 调用视为复杂的社交网络,并应用基于社交网络的中心性分析来挖掘调用图中的中

心节点, 计算敏感 API 调用与中心节点之间的平均亲密度来表示图的语义特征进行检测, 但算法依靠领域专家构造敏感 API 列表, 模型的更新难以跟上恶意软件更新换代的速度。

Ma 等人^[46]提出了一种使用静态分析方法构建按时间顺序排列的 API 数据集并使用 LSTM 分析的检测方法, 从应用程序的控制流图中提取 API 调用, 并基于 API 信息构建了布尔型、频率型和时间序列型数据集, 对应 API 调用、API 频率和 API 序列三个方面, 并依据这三个数据集分别构建恶意软件检测模型进行检测, 通过软投票方法给出最终检测结果。该方法说明 API 序列的语义特征和结构特征在识别恶意软件上都起到了重要作用; Agrawal 等人^[47]提出了一种基于动态分析的两阶段模型, 该模型以软件运行时的 API 序列为样本, 通过添加对 API 调用参数的分析提高检测性能。

当前恶意软件检测技术还有一些亟待解决的问题, Saad 等人^[48]通过对近些年相关研究的分析, 总结归纳出当前恶意软件检测所需要解决的问题:

(1) 训练检测模型的时间成本和人力成本高。一些模型所需要的数据特征规模大和使用复杂模型, 如 Raff 等人^[43]以整个可执行文件为特征, 导致模型训练时间成本高; 此外, 一些模型训练需要的数据特征依靠人工构造, 这就导致了在时间和人力上的高成本。

(2) 基于深度学习的恶意软件检测模型的可解释性差。深度学习本身就是黑盒模型, 内部运行原理并不明晰, 在实际训练中需要不断优化调参, 对输入与输出之间的联系缺乏明确的解释。

(3) 针对对抗性恶意软件样本的检测研究不足。对抗性恶意软件样本旨在通过特征隐藏、混淆、加壳、反虚拟等技术绕过传统的恶意软件检测机制。对其进行检测需要更加先进的方法。

(3) 恶意软件检测模型的准确率、鲁棒性表现不佳。当前恶意软件更新速度快, 导致传统的恶意软件检测模型面对新型恶意软件, 缺少对应特征导致准确率不佳; 此外, 一些良性软件由于自身功能需求会进行敏感操作, 如涉及到广告功能一般需要访问用户敏感信息, 使这些灰色行为使得检测模型难以分辨以至于出现误报问题。

面对当前恶意软件检测技术的不足, 本文针对恶意软件检测算法检测准确率和鲁棒性表现欠佳的问题, 通过结合 API 语义特征和结构特征进行检测, 增加特征包含的信息, 以提高模型的准确率和鲁棒性; 针对算法模型更新速度慢的问题, 通过固定特征的维数, 控制训练所需数据集的规模, 自动化特征提取和构造过程, 来加快检测模型的更新速度。

1.3 研究内容

恶意软件给网络空间安全带来了巨大挑战,许多研究人员致力于研发高效可行的恶意软件检测技术,并且已经取得了许多显著的成果。当前已有的恶意软件检测方法各有优缺点。恶意软件无论如何隐藏其特征,最终都要依靠 API 函数去执行特定的行为获取操作系统内核资源,因此研究恶意软件运行时 API 序列对于提升恶意软件检测性能具有重要意义。

由于开放性、兼容性和广泛的使用,Windows 操作系统一直是恶意软件攻击的主要目标。可执行文件(exe 文件)是一种可在 Windows 系统存储空间中浮动定位的可执行程序,它可以加载到内存中并由操作系统加载程序执行,被用来安装或运行软件应用程序,也是恶意软件的主要形式之一。

本文对 Windows 系统上的可执行文件运行时调用的 API 序列从不同角度进行综合分析,提高当前恶意软件检测算法准确率、抗干扰能力、鲁棒性,提出了新型恶意软件检测模型,并通过实验对模型性能进行了全面验证。另外,本文使用的数据集以 Windows 平台上的可执行文件通过在沙箱中运行获取的日志中提取的 API 序列为样本。下面是本文的主要研究内容:

(1) 提出了一种新的恶意软件检测模型 Mal-CLAM。该模型使用 API 序列作为特征,通过结合 API 序列的上下文依赖关系和结构特征进行检测。模型采用多个门控卷积神经网络从不同维度提取 API 序列的深层特征,利用双向长短期记忆网络进一步提取恶意软件的上下文依赖关系;使用深度自编码器对 API 序列压缩重构获取重构误差,使模型能够学习其结构特征,捕获恶意软件的结构特征;将获取的 API 序列结构特征和依赖关系融合,利用神经网络中的多层感知机进行检测,增强了检测性能,而且降低模型更新对专家知识的依赖,最大限度地减少了人工参与,加快模型更新需要的时间,提高了整体检测效果。此外,本文还通过消融实验探究了模型中每个模块对整体检测性能的影响,和其起到的作用,为 Mal-CLAM 模型各模块参数选取提供切实可行的依据;通过对比实验,将 Mal-CLAM 模型与现有的恶意软件检测模型进行了比较,结果表明所提出的模型在检测准确率、抗干扰能力上超过了其他模型。

(2) 基于本文提出的 Mal-CLAM 模型搭建了恶意软件检测系统,包括登录注册模块、恶意软件检测模块、模型更新模块、历史记录搜索模块和可视化模块。检测系统通过网站形式提供服务,适用于当前主流浏览器,针对 Windows 系统上的可执行文件提供高效、准确的恶意软件检测服务。检测时将用户输入的可执行文件放到安全虚拟环境中运行获取日志,从日志中提取 API 序列,之后将序列输入到模型中获取检测结果。为了让检测结果更加可靠,除了 Mal-CLAM 模型

外，还引入其它四种恶意软件检测模型共同检测。

1.4 论文组织架构

本论文共包括五篇章，具体组织架构如图 1-2 所示，各部分具体内容如下：

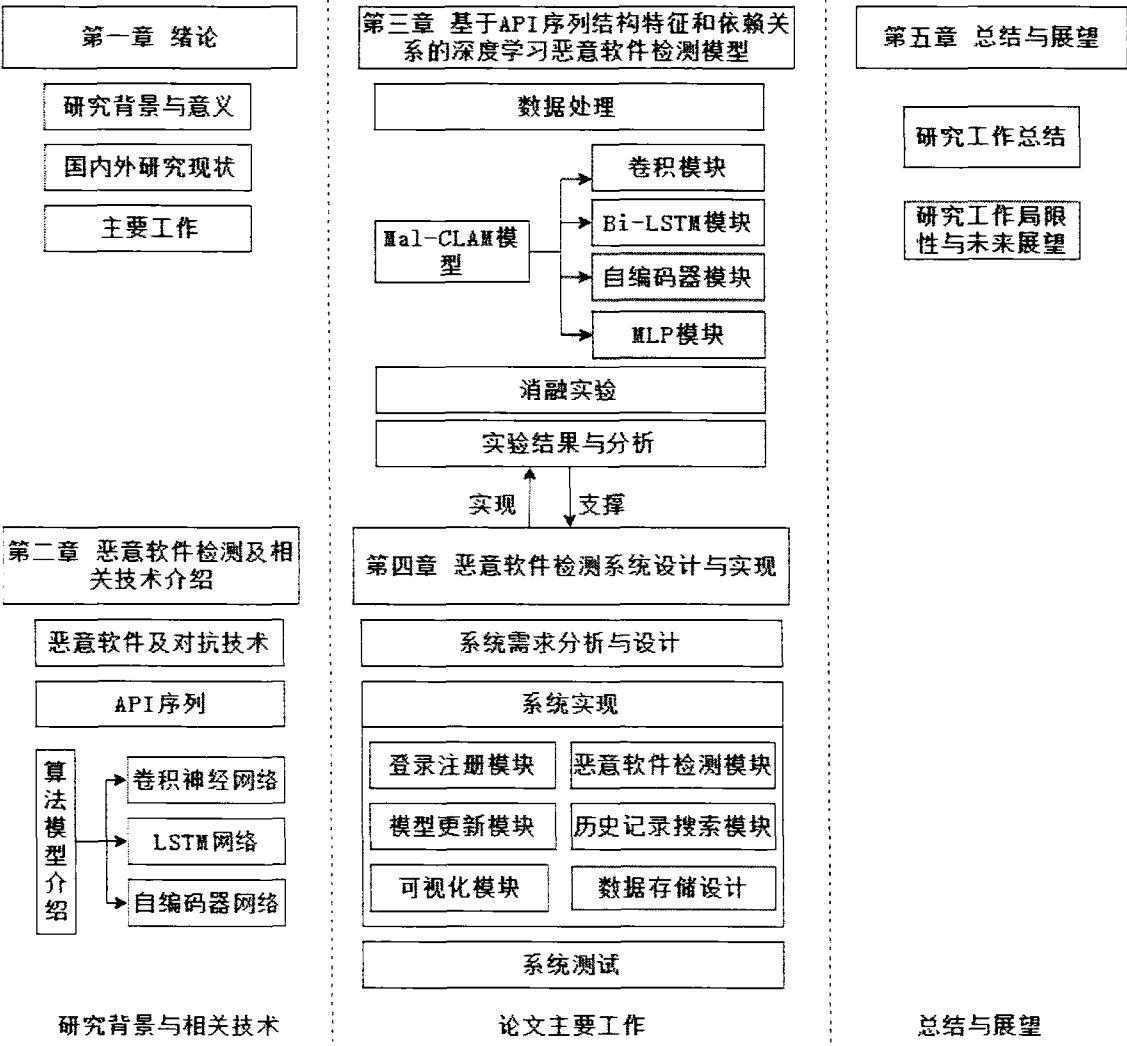


图 1-2 论文组织架构

第一章是绪论。主要介绍选题的背景和意义，介绍了当前网络空间面临的安全挑战和恶意软件检测所面临的困难；分析了国内外对恶意软件检测方法的研究状况；最后对本文进行的先期研究进行了简要描述，对论文的组织结构进行了检验介绍。

第二章对恶意软件概念和相关技术进行阐述。首先讲述了恶意软件的基本概念、然后讲述了最常见的类型和典型的样例，常见的躲避检测方法以及最新的对抗技术；接着阐述了 API 序列的知识，以及对恶意软件测试的重要性；最后介绍了中应用到的算法模型，包括机器学习模型和深度学习模型的原理和用法进行了讲解。

第三章设计并实现的基于 API 序列特征和依赖关系的深度学习恶意软件检测模型 Mal-CLAM 模型，介绍了使用的数据集和特征处理过程，Mal-CLAM 模型的整体框架和流程，详细描述各个模块的功能和原理。然后分析和评估了实验结果，首先介绍了实验的前期准备工作，如使用的数据集、评价标准和参数设置；进行消融实验研究了各个模块对整体检测性能的影响，获取最佳构造；进行实验验证模型的检测表现；通过对数据集样本标签的设置验证了模型具备一定程度上的抗干扰能力。

第四章介绍了依据 Mal-CLAM 模型实现的恶意软件检测系统，进行了需求分析、结构设计和开发实现。具体介绍了系统包含的功能和实现原理，进行了全面的测试，并配有相关的实际使用情况展示。

第五章对本文的研究工作进行一个全面的总结归纳，指出了当前工作还存在的不足与待解决的问题，最后对未来的研究方向进行了展望。

第二章 恶意软件检测及相关技术介绍

本章主要介绍了与恶意软件检测工作相关的理论知识,包括对恶意软件的定义,恶意软件常见的种类以及目前常见的恶意软件对抗技术;然后介绍了本文的主要研究对象,API 调用序列的相关概念;本章最后对研究使用的模型算法进行原理介绍,方便后续的理解。

2.1 恶意软件及对抗技术

2.1.1 恶意软件定义及类别

恶意软件(Malicious Software, Malware)是指攻击者为达到各种目的而开发的软件代码或计算机程序^[49],通过恶意行为来破坏计算机系统并损害用户的合法权益。这些恶意软件通常通过网络、操作系统以及安全漏洞等方式进行传播。由于大多数用户缺乏安全防护意识和技能,导致恶意软件得以隐匿并攻击用户的计算机。恶意软件对网络空间的攻击和破坏十分严重,许多恶性事件都是有恶意软件引起的。

根据不同的攻击意图,恶意软件可分为病毒、蠕虫、勒索软件、木马、后门程序等,其在形式上也是多种多样的,包括脚本文件、可执行文件、插件等。近些年来,恶意软件的攻击范围也逐步扩大,如利用云计算技术和物联网系统将潜在的恶意软件感染位点从计算机扩展到任何电子设备。具体内容如下:

病毒主要用于攻击计算机系统^[50],可以在未经授权的情况下,通过感染计算机系统盗取数据、干扰系统的正常运行,它们可能会损害文件系统、篡改数据、窃取敏感信息,甚至完全瘫痪整个系统。计算机病毒可以通过网络、可移动存储设备或通过潜在的安全漏洞等方式传播。

蠕虫是一种能够自我复制的恶意软件^[51],主要通过寻找系统存在的、当前未被发现并修复的未公开漏洞进行不断地传播,主要包括 Windows 系统漏洞、安卓系统漏洞以及网络服务器漏洞等开源、兼容性高、利益价值大的系统存在的漏洞。蠕虫不需要人工干预,具有较强的独立性,它能够利用系统存在的漏洞主动出击,执行恶意攻击损害计算机系统。被感染的计算机会在蠕虫的攻击干扰下出现各种问题,包括系统运行越来越缓慢、文件出现丢失现象、文件或程序被破坏无法正常打开或使用、出现新的无意义文件的情况。蠕虫的传播和攻击行为是基于程序自身设计的,不受宿主主机的限制,因此蠕虫的传播速度很快,可以实现大规模感染和传播。

勒索软件^[52]会锁定受害者的数据或设备,并威胁受害者,使其保持锁定状态

(或更糟糕的状态),并要求受害者向攻击者支付赎金。近些年来,勒索软件攻击已经演变为包括双重勒索和三重勒索攻击。

计算机木马^[53]名称来源于古希腊神话中的特洛伊木马,希腊联军将一只巨大的木马送给特洛伊城,城中的人们将其带入城内,但实际上木马内藏有隐藏的士兵,导致了城市的沦陷。计算机木马就是通过诱骗使用者,在使用者不知情的前提下进入其计算机系统,然后实施各类恶意行为,如盗取敏感信息、损坏文件系统、制造后门以及散播其它恶意软件,具有自启动、自我恢复能力、隐蔽性强。

后门程序可以躲避系统安全控制系统^[54],其主要目的是为黑客提供远程访问和控制系统的权限,而不是直接破坏系统或窃取数据。攻击者会通过远程访问的方式进入受感染的计算机或系统,并获取完全的控制权,以便进行各种恶意行为。后门程序的入侵方法多种多样,包括通过漏洞利用、社会工程学攻击、远程文件包含等方式。一旦安装成功,后门程序就会成为黑客用来进行网络攻击的平台。

2.1.2 恶意软件对抗技术

恶意软件对抗技术^[55]是指恶意软件隐藏特征、躲避检测与查杀的一系列方法。随着人工智能的兴起,恶意软件衍生变体的速度越来越快、躲避检测的方法越来越多,给恶意软件检测工作带来了许多挑战。目前常见的对抗技术有:

加壳技术:加壳一种压缩、加密或修改恶意文件格式,改变其特征形态的技术。加壳后的恶意软件能够保证壳内程序正常运行的情况下,躲避大部分的基于静态特征的检测,增加恶意软件的网络中的存活时间和传播能力。以下是一些常见恶意软件加壳技术:

(1) 静态加壳技术:攻击者直接修改二进制文件的结构,将恶意软件的代码嵌入到一个外部的壳程序中,这会改变程序的入口点,以便在运行时首先执行壳程序,然后再解压和执行恶意代码。

(2) 动态加壳技术:恶意软件的原始代码在存储介质上以加密的形式存储,在运行时会自动解压并解密其代码,动态加壳通常会使得静态分析难以获取恶意软件的原始代码。

(3) 虚拟机加壳技术:恶意软件的原始代码被转换为虚拟机指令,在运行时使用虚拟机技术来执行恶意代码,这使得恶意软件更难以被分析和检测,因为分析者需要了解虚拟机的运行时环境。

混淆技术:混淆技术是一种用于隐藏、困扰和增加恶意软件分析难度的方法。通过混淆,攻击者可以使分析人员更难以理解、逆向工程和检测其恶意代码,以到达躲避静态分析、逆向分析等检测方法。以下是一些常见的恶意代码混淆技术:

(1) 代码重命名:恶意软件开发者对变量名、函数名等标识符进行随机替换、或者使用无关语义重命名,使代码变得难以理解和分析。

(2) 无用指令插入: 开发者通过向恶意软件中添加大量无意义的代码片段, 在不影响原本功能的前提下增大代码体积, 增加代码分析和反向工程的难度。

(3) 数据混淆: 通过加密或着编码处理, 将恶意代码中的常量、字符串进行处理, 加大检测人员分析理解的难度和工作量。

(4) 指令混淆: 在汇编语言层次对恶意攻击机器码进行变化或替换, 使得原始的指令序列难以识别, 增加分析的难度。

反虚拟技术: 反虚拟技术是用于对抗检测和虚拟化环境的方法。在动态分析中, 通常会搭建安全虚拟环境, 如沙箱, 来分析软件运行时的特征。利用反虚拟技术, 恶意软件会试图识别当前运行环境, 一旦发现异常就会停止攻击或正常运行。以下是一些常见的反虚拟技术:

(1) 时钟偏移检测: 由于虚拟机的指令执行速度通常会比物理机慢, 因此恶意软件可以通过测量指令执行的时间来检测虚拟机的存在。

(2) 硬件资源审查: 恶意软件通过检测运行环境中的硬件设备, 如虚拟网卡、虚拟磁盘等, 并利用这些信息来判断自己是否处于虚拟化环境中。

(3) 异常检测: 由于虚拟环境和正常环境对异常的处理机制不同, 恶意软件可以利用异常处理机制来检测虚拟机中的异常行为, 如异常中断、异常指令等。

当前, 随着神经网络、自动化等技术的发展, 很多系统的安全防护措施进一步加强, 对恶意软件的检测和防御能力也进一步提高。但是, 同样的技术同样也可以被攻击者运用于批量、快速地开发新的恶意软件以及其变体上面, 并且由于恶意软件检测属于被动防御的性质, 天然就带有滞后性, 因此新时代的网络空间安全形势会变得越来越严峻, 除了技术更新, 其它方面, 诸如完善管理、加强立法、风险隔离等手段和方法也要不断更新, 跟上时代步伐。

2.2 API 调用序列

应用程序接口 (Application Programming Interface, API) 是应用程序接口, 是系统内核进行资源管理、权限控制的函数, 是 Windows 系统中第三方机构开发的应用程序与操作系统交互的桥梁。

在 Windows 操作系统中, 出于安全方面的考虑, 许多内核中资源是不能够被用户直接调用的, 如进程、网络、文件等。用户想要进行上述操作就需要调用 API 来实现。通常情况下, 软件的执行都需要或多或少的调用一系列的 API, 而这些 API 序列则能够包含许多重要信息。一般恶意软件进行攻击都会通过 API 序列去对敏感的资源进行操作, 如通过 RegOpenKey、RegSetValueA、RegCloseKey 三个 API 实现修改注册表的行为。不同的攻击方法调用的 API 序列不同, 而且通常是一些较少出现的序列。因此通过 API 序列来进行恶意软件检测具有很好

的发展前景。表 2-1 列出了一些恶意软件频繁调用的 API。

表 2-1 恶意软件常用 API 调用

类型	API	功能
文件和注册表操作	CreateFile	创建或打开文件
	WriteFile	向文件写入数据
	ReadFile	从文件读取数据
	RegCreateKey	创建注册表键
	RegOpenKey	打开注册表键
	RegSetValue	设置注册表值
	RegQueryValue	查询注册表值
系统信息获取	GetSystemInfo	获取系统信息
	GetComputerName	获取计算机名称
	GetUserName	获取当前用户名称
进程线程控制	CreateProcess	创建新进程
	OpenProcess	打开已存在的进程
	CreateThread	创建新线程
内存操作	VirtualAlloc	分配虚拟内存
	WriteProcessMemory	在其他进程中写入内存
	ReadProcessMemory	在其他进程中读取内存。

相比于静态分析易受到加壳技术和混淆技术的干扰，动态分析方法通过在安全虚拟环境中运行恶意软件，从运行日志中获取 API 序列，更为准确和可靠。API 序列在动态分析中起着至关重要的作用。Taheri 等人^[56]收集并发布了一个恶意软件数据集 CICAndMal2017，该数据集包括了软件的静态特性的权限和意图，以及作为动态特性的 API 调用，通过恶意软件分析器来检查这些特征，实现了基于动态分析的恶意软件类别分类的 83.3%的精确度和基于动态分析的恶意软件家族分类的 59.7%的精确度；Chen 等人^[57]采用基于规则和基于聚类的分类方法来评估参数对恶意行为的敏感度进行检测，并根据不同敏感度的运行时参数进一步标记 API 调用，然后通过连接标记 API 的本机嵌入和敏感嵌入来对 API 进行编码，用于表征连续标记 API 之间的关系及其在安全语义方面的对应关系，最后

将嵌入的 API 序列输入深度神经网络，以训练二元分类器；Azadeh 等人^[58]提出了以软件操作码、二进制文件信息、API 调用的统计信息等综合信息作为软件特征，基于 N-gram 算法和 Top K 算法在新的未知文件的分类中选择最相似的 K 个文件计算相似度进行检测的方法。

然而，大多数现有方法倾向于忽略 API 序列中的依赖关系。一些方法将 API 及其参数视为仅字符串，而其他方法只关注统计信息。因此，API 序列的全部潜力仍未得到充分利用。为了从 API 序列中提取深层特征，许多研究深入探索了机器学习和深度学习方法领域。机器学习（ML）算法，如 KNN、NB、DT 和 SVM，在 API 序列分析^[59]中得到了广泛的应用。近年来，深度学习在自然语言处理^[60]和计算机视觉^[61]方面取得了相当不错的研究成果，促进了端到端模型的发展，显著减少了人工干预，提高了性能。因此，将深度学习集成到恶意软件检测领域是一项至关重要的工作。许多基于深度学习的方法已被用于分析 API 序列^[62]。Luxiaofeng 等人^[24]采用随机森林模型和 LSTM 模型，将恶意软件的静态特征与从 API 序列派生的统计特征相结合进行恶意软件分类，但是这种方法忽略了 API 序列中相邻 API 之间的互连，只关注统计特征；另一方面，Zhang 等人^[61]聚焦在 API 参数对恶意软件检测的影响，通过丰富使用 API 参数以丰富 API 序列包含的信息；Catack 等人^[64]利用嵌入层和 LSTM 模型来理解 API 序列中的关系。尽管如此，他们的方法采用了单向 LSTM 模型，该模型只考虑当前 API 与过去调用的 API 之间的关联，而忽略了与未来 API 的联系。

随着各种抗检测和混淆技术的发展，恶意软件始终努力逃避跟踪和检测，仅依靠单个签名使得准确识别现实世界的恶意软件具有挑战性。因此，从 API 序列中全面提取多种类型的内在特征对于提高恶意软件检测性能十分重要的。

2.3 恶意软件检测技术

本节对本文提出的恶意软件检测模型中涉及到的算法进行简要介绍，主要包括卷积神经网络、长短时记忆网络和自编码器。

2.3.1 卷积神经网络

卷积神经网络（Convolutional neural network, CNN）是一种专门用来处理具有复杂网络结构信息的深度学习模式^[65]，比如图像、视频和声音等。随着科研人员的不断研究，取得了令人瞩目的研究成果，并在生产、生活、军事等各方面被广泛应用^{[66][67]}。CNN 包括卷积层、池化层、激活函数、全连接层等结构，每个结构的具体功能如下：

卷积层（Convolutional Layer）：卷积层依靠卷积操作来处理输入数据并提取出其携带的局部特征。卷积操作通过卷积核在输入数据上移动，计算局部区域与

滤波器的卷积，从而得到特征图。卷积核的重要参数是卷积核的尺寸和数目。

池化层 (Pooling Layer)：池化层对特征进行降维，节省计算资源，提高泛化能力。常见的方法包括最大池化法和平均池化法。最大池化法 (Max Pooling) 顾名思义就是在需要过滤的区域内通过数值对比选择其中的最大值。平均池化 (Average Pooling) 是在过滤的区域内加和求平均，降低冗余信息的干扰。

激活函数 (Activation Function)：激活函数通常位于在卷积层后面，以引入非线性变换，使得神经网络能够学习和适应各种复杂的数据模型。卷积仅仅实现线性变换，因此不能够表达出更丰富和复杂的语义，因此需要使用激活函数对卷积的结果进行非线性映射，增强模型的表达和学习能力。

全连接层 (Fully Connected Layer)：全连接层负责将从图像提取的局部特征整合在一起，用于进行分类或其他任务。全连接层的每一层的所有单元都与上一层完全连接，然后通过矩阵乘法进行特征空间的变换，将有用的信息进行非线性组合得到输出。在实际使用中，可以依据具体需求和运行资源情况自由设置全连接层的数量，通常会设置一两个，最后一层通过激活函数得到分类结果。

卷积神经网络可以根据输入数据的维度分为 1D-CNN、2D-CNN 和 3D-CNN。1D-CNN (一维卷积神经网络) 主要用于处理序列数据，如时间序列数据、文本等；2D-CNN (二维卷积神经网络) 主要用于处理图像数据，如图像分类、目标检测等任务；3D-CNN (三维卷积神经网络) 主要用于处理带有时间维度的体积数据，如视频数据，可用于视频分类、动作识别等任务。本文的研究目标是处理好的 API 调用序列，因此使用 1D-CNN 模型。

卷积神经网络已经被用于恶意软件检测并取得了不错的成果，如 Amer 等人^[68]构建了一个名为多尺度特征提取和融合网络的模型，创建了描述恶意和非恶意过程的通用行为图模型，依赖于统计、上下文和图形挖掘功能的融合，捕获调用序列中 API 函数之间的显式和隐式关系，并使用 1D-CNN 进行恶意软件检测，准确率达到了 94%。

2.3.2 长短时记忆网络

长短时记忆网络 (Long Short-Term Memory, LSTM) 专门用于处理序列数据和时间序列数据，它的本质是一种深度学习中的循环神经网络 (Recurrent Neural Network, RNN) 变体^[69]。LSTM 模型相比其它模型具备很强的记忆能力，能够有效地处理长序列数据，如文本、语音和时间序列。它在许多应用中表现出色，包括自然语言处理 (文本生成、情感分析、机器翻译)、语音识别、时间序列分析 (股票预测、天气预测) 等。

LSTM 是为了解决传统 RNN 中的梯度消失问题而设计的，它具有记忆单元和门控机制，可以有效地捕捉和记忆长期依赖关系。Xi 等人^[70]将一个系统调用

序列视为一个语句模型进行语义分析，构建基于长短期记忆（LSTM）语言模型的分类器；在分类器中，首先分别使用来自恶意软件和良性应用程序的系统调用序列来训练两个应用不同场景的 LSTM 模型；然后根据这些模型的输出结果，计算两个相似度分数；最后，分类器根据较高的分数确定所分析的应用程序是恶意的还是可信的。Mathew 等人^[71]训练了 RNN-长短期记忆（LSTM）模型，根据术语频率-逆文档频率（TF-IDF）推荐特征的相对排名，从 API 序列数据集中最具信息量的序列中学习，实验结果表明模型能够从未知的测试 API 调用序列中检测恶意软件和良性软件。这些研究工作说明 LSTM 在恶意软件检测方面是大有可为的。

LSTM 的模型结构如图 2-1 所示：

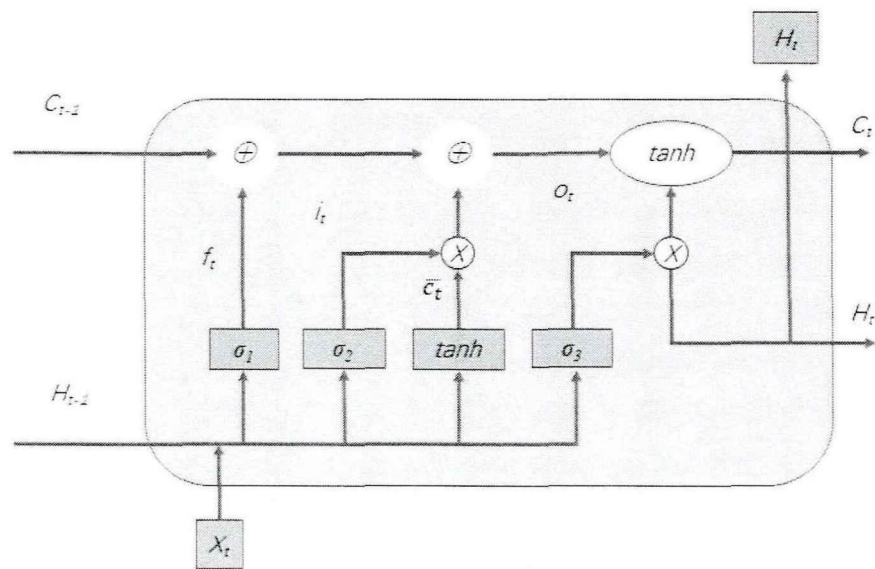


图 2-1 LSTM 模型结构

LSTM 模型由单元状态、遗忘门、输入们、更新单元状态、输出们五部分组成，可以在运行时根据获取的数据特征，自主选择遗忘或更新，具体功能如下：

单元状态（Cell State）：LSTM 中的核心是单元状态，它是一个线性传递的记忆单元，可以捕获和存储有关序列中过去信息的长期依赖关系；

遗忘门（Forget Gate）：遗忘门能够控制是否从单元状态中舍弃某些信息。它使用一个 Sigmoid 激活函数来输出一个 0 到 1 之间的值，其中 1 表示完全保留，0 表示完全遗忘，可以看作是该信息的权重；

输入门（Input Gate）：输入门允许新信息流入单元状态。它由两部分组成，Sigmoid 函数确定更新的值，Tanh 函数创建新候选向量；

更新单元状态（Update Cell State）：在遗忘门和输入门的控制下，单元状态被更新。部分信息被遗忘，部分新信息被添加；

输出门（Output Gate）：输出门确定单元状态的哪些部分将被输出到下一个

时间步；输出门使用 Sigmoid 函数和 Tanh 函数来计算输出。

2.3.3 自编码器

自编码器是一个无监督的神经网络模型^[72]，分为解码和编码过程。在编码过程中，自编码器学习获取丢失其实际意义的潜在特征；解码是学习如何基于编码特征构建数据以学习如何构建数据的过程，使其尽可能接近原始数据。自编码器由神经元组成，其具体结构如图 2-2 所示：

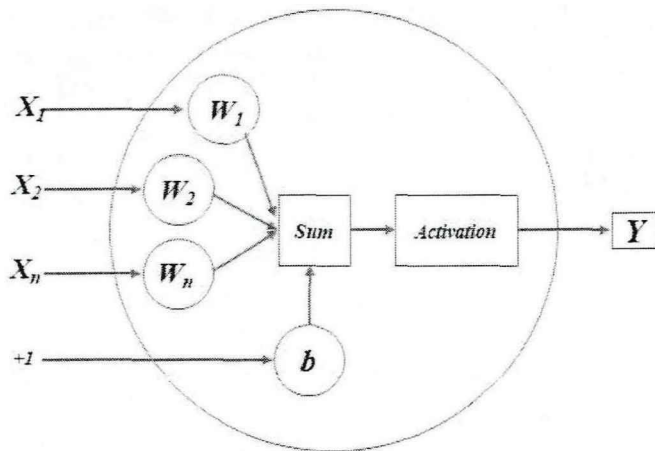


图 2-2 神经元结构

当训练神经网络时，每一层的每个神经元权重，并根据集合权重对上层每个神经元传输的值求和，然后将它们传递给激活函数，经过非线性变换后的输出到下一层（隐藏层或输出层）。激活函数用于揭示输入特征之间的非线性相关性。如果没有使用激活函数，输出是输入值的线性处理的结果，这失去了它的有用性。

激活函数用来揭示输入特征之间的非线性相关性，如果不使用激励函数，则神经网络中的各个层，各种架构，都会失去应有的作用，因为这样的输出都是输入数值的线性处理结果，没有什么意义。在实际的实验中常用 sigmoid 函数，relu 函数，tanh 函数等。

深度自编码器具有多层的编码和解码结构，通过堆叠多个隐藏层，能够学习更复杂的数据特征。这种模型广泛用于特征提取、降维和生成新数据。深度自编码器在图像、文本和语音等领域取得了显著的成果，为数据表示学习提供了一种强大的工具。

目前，自编码器已被用于恶意软件检测并且得了良好的成果。Xing 等人^[40]将恶意软件的灰度图像表示与自编码器网络相结合，分析了基于重建误差的恶意软件灰度图像方法的可行性，使用自编码器实现了恶意软件检测；Wang 等人^[73]构建了 DAE-CNN 模型，将 API 序列构造为稀疏矩阵，使用深度自编码器作为 CNN 的预训练方法，利用自编码器从这些矩阵中提取最具代表性的判别特征进行恶意软件检测，达到 90% 的精度。

第三章 基于 API 序列的深度学习恶意软件检测模型

本章提出了一种基于 API 序列结构特征和依赖关系的深度学习恶意软件检测模型—Mal-CLAM 模型。本章说明了研究解决的问题挑战,介绍了恶意软件检测流程框架;对模型的结构和原理进行说明分析,模型包括卷积模块、Bi-LSTM 模块、自编码器模块和 MLP 模块;通过消融实验探究了各个模块在检测中起到的作用,并进行模型结构调整和优化;最后设计并进行实验,验证了本文方法的有效性,提出的恶意软件检测模型达到了 94.49%的准确率、92.75%的召回率、93.63%的 F1 值和 98.43%的 AUC 值。

3.1 问题分析和模型框架

3.1.1 恶意软件检测技术问题分析

尽管当前对基于深度学习的恶意软件检测方法的研究取得了不少成果,但仍有许多问题需要解决。Saad 等人^[48]指出当前使用深度学习的检测算法对恶意软件和良性软件的特征识别不够精确,容易出现误报问题。许多基于动态分析的方案使用 API 的语义进行检测,通常涉及到敏感资源的操作会被定性为恶意。一些良性软件由于自身功能需求也会进行同样的敏感操作,如涉及到广告功能一般需要访问用户敏感信息,如相册、浏览记录、录音,甚至调用摄像头监控用户的行为,以获取广告利润,使这些灰色行为使得检测模型难以分辨以至于出现误报问题;Raff 等人^[43]的研究使用了深度卷积神经网络进行检测,虽然检测效果良好,但是模型过于庞大和复杂,模型更新速度慢,训练一次需要耗费 180 小时左右,耗费的计算资源也很大。恶意软件在不断推陈出新,许多恶意软件检测算法由于不能及时更新,导致缺少对新型恶意软件的了解与认知,从而使检测结果变得不准确、不可靠。

针对恶意软件检测算法检测准确率和鲁棒性表现欠佳的问题,本文在研究中通过增加特征包含的维度,如结合 API 语义特征和结构特征,从多方面提取检测特征以提升模型的表现;通过消融实验剖析检测模型的组成部分,深入理解各部分的作用,为模型的优化与改进提供指导。

针对算法模型更新速度慢的问题,本文使用 API 调用序列作为特征,在保证准确率的情况下,固定了特征的维数,控制了训练所需数据集的规模,且特征构建过程无需人工参与,缩短了提取特征所需要花费的时间,因此可以更快速的更新模型。

3.1.2 恶意软件检测模型框架

本文针对当前恶意软件检测算法准确率低、鲁棒性不强，模型更新速度慢等问题，提出了一个恶意软件检测模型框架。其分为三个部分：软件和行为特征收集部分、特征提取部分、检测和训练部分。具体的模型框架如图 3-1 所示：

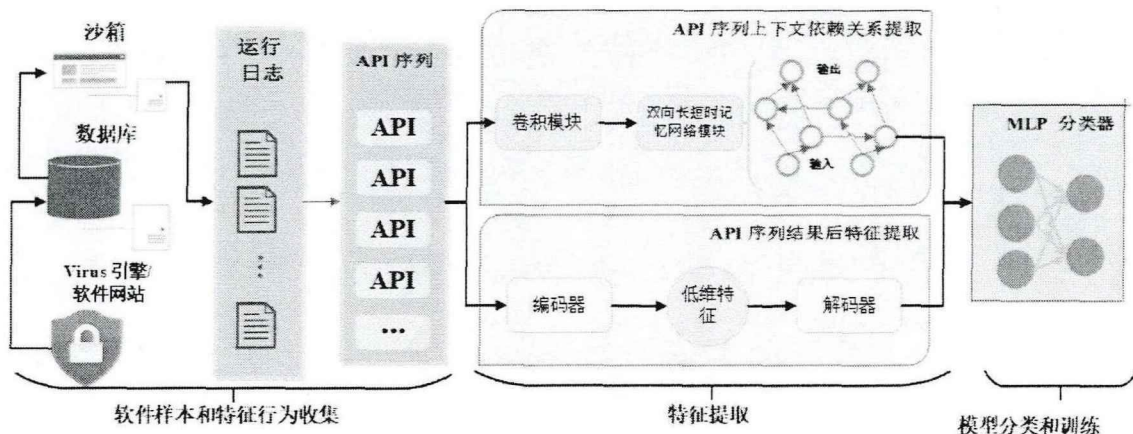


图 3-1 恶意软件模型框架

(1) 软件和行为特征收集部分。首先从开放网站上获取恶意软件和良性软件的可执行样本；然后在沙箱中运行软件获取运行日志；最后编写程序从日志中获取软件运行的 API 序列。由于恶意软件为了躲避检测，经常会在攻击期间插入冗余操作，使得获得的 API 序列中存在大量的重复的 API，降低了序列有效信息密度，干扰最后的检测结果，因此对 API 序列进行了去重操作；此外，各个软件的体量大小和运行逻辑不同，执行不同的任务需要调用的 API 序列长度也不尽相同，甚至会有几十倍的长度差异，会干扰检测结果。为了统一样本序列的长度，本文中每个软件样本对应的 API 序列长度设置为 1000，过长的序列舍弃后面的 API，太短的序列则添加空操作进行补全。通过固定了特征的维数在一个可接受的范围内，缩短了特征提取过程耗费的时间，加快模型更新速度。

(2) 特征提取部分。特征提取部分为两个模块：一是对 API 序列之间的依赖关系的提取，API 序列实际上是一段前后相关联的、有先后关系的语言文本，类似于自然语句，因此通过门控卷积网络和 Bi-LSTM 两个模块来获取其中的关键信息特征；二是对 API 序列的结构特征的提取，恶意软件和良性软件所执行的整体操作性质不同，在选取的 API 函数、数量种类上必然存在一定的差异，通过深度自编码器对序列进行压缩重构来获取 API 之间深层特征作为检测的依据。通过结合 API 调用序列的结构特征和上下文依赖关系，让模型学习到更全面、更丰富、更泛化的恶意软件的行为特征，提升检测模型的准确率、降低误报率，增强模型鲁棒性。

(3) 检测和训练部分。将 API 调用序列的上下文依赖关系特征和序列的结

构特征连接起来,使用 MLP 模型进行计算,通过遗忘 MLP 密集层之间的连接,使模型学习更泛化的特征,防止过拟合现象、提高模型泛化能力。模型最终输出一个概率值,即输入样本是恶意软件的概率,当概率高于阈值时即判定其为恶意软件;然后将实验模型的检测结果与其它的恶意软件检测模型进行比较,从准确率、召回值、AUC 值、F1 值等多方面进行分析,证明了本文提出的 Mal-CLAM 模型的检测性能的优越性。

本文提出的恶意软件检测框架,通过软件和行为特征收集部分快速获取 API 调用序列作为检测特征;在特征提取部分使用深度学习模型提取 API 序列的结构特征和上下文依赖关系;在检测与训练部分,使用多层感知机进行检测,使结果更准确、模型可靠。

3.2 基于 API 序列结构特征和依赖关系的恶意软件检测模型

本节针对 3.1 提出的问题,介绍了提出的基于 API 序列结构特征和依赖关系的深度学习恶意软件检测模型—Mal-CLAM 模型,利用深度自编码器对 API 序列的压缩重建计算重构误差获取结构特征;利用三个不同步幅的门控卷积网络提取 API 序列中的局部关系,并进一步利用 Bi-LSTM 模型捕获 API 序列之间的上下文依赖关系;将两部分特征融合利用 MLP 网络对而已软件进行识别检测。

3.2.1 Mal-CLAM 检测模型架构

Mal-CLAM 模型包括卷积模块、Bi-LSTM 模块、深度自编码器模块和 MLP 模块四部分,具体架构如图 3-2 (a) 所示:

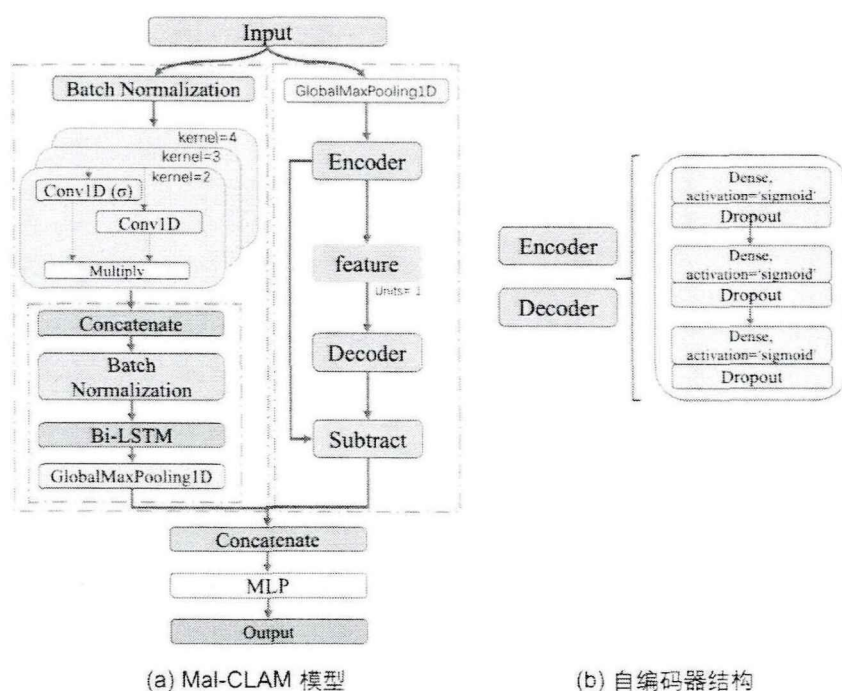


图 3-2 Mal-CLAM 模型架构

输入模块：模型的输入数据是固定长度为 1000、经过去重处理的 API 调用序列。

卷积模块：该模块采用多个门控卷积网络，通过选择性的遗忘部分特征的机制解决了传统卷积把输入像素一视同仁的问题，有效降低了 API 序列中无效信息作的干扰，高效地提取 API 序列中的局部关系。该模块使用三个不同步长的门控卷积核，类似于不同长度的滑动窗口起到的作用，从不同维度上提取 API 序列的高级特征表示。

Bi-LSTM 模块：将 API 序列视为自然语言文本，卷积模块提取其中的高级特征，然后进一步通过两个独立的 LSTM 网络处理前向和后向的顺序数据，捕获其上下文关系；前向 LSTM 从头到尾处理序列，而后向 LSTM 从尾到头处理序列；然后连接两个网络的输出以产生最终预测。相比单个 LSTM 模型能够处理更长的 API 序列，获得更多的信息；

深度自编码器模块：编码器将输入数据压缩成低维的编码，解码器则将该编码恢复成原始数据，并通过不断优化重构误差来优化模型。恶意软件和良性软件的特征不同使用，因此构建的重构误差也不同可以用来进行检测。通过这种方式，学习输入数据的内在结构，获取 API 序列的内在深度特征。同时，由于自编码器是无监督神经网络，无需人工构造特征，可以大大减少领域专家的参与。

MLP 模块：即多层感知器^[74]，由多个密集层组成，具有较强的拟合能力和灵活性，可以拟合复杂的函数解决分类问题，适用于不同类型的数据和问题，它被广泛用于各种深度学习任务，如分类、自然语言处理等。本文模型使用它接收 Bi-LSTM 模块和深度自编码器模块的特征并连接起来进行检测，通过神经元之间的特征传播和权重值更新，学习输入特征与输出结果之间的复杂关系，用来判定软件样本是否为恶意。

3.2.2 基于门控卷网络的特征选择

1D-CNN^[75]擅长处理顺序数据中的局部关系，在语音识别、自然语言处理和时序序列预测等任务中产生卓越的性能。1D-CNN 主要用于处理一维序列数据，如音频和文本。与传统的全连接神经网络相比，1D-CNN 可以更好地处理序列数据中的局部关系，因此在语音识别、自然语言处理和时序序列预测等任务上表现更好。1D-CNN 使用卷积层从顺序数据中提取特征。卷积层通过滑动固定大小的窗口对输入数据执行卷积操作，提取窗口内的特征，然后将这些特征映射到下一层，使用池化层来降低特征图的维数和计算工作量。卷积运算公式如(3-1)所示：

$$X[i] = \text{activation}(\text{convolution}(\text{input}[i:i+k], \text{kernel}) + \text{bias}) \quad (3-1)$$

其中， $X[i]$ 表示新生成的特征的 i 个元素， activation 表示非线性激活函数， $\text{input}[i:i+k]$ 表示输入数据中第 i 个元素到第 k 个元素的序列， kernel 表示卷

积核, bias 表示偏置项。

1D-CNN 的具体结构如图 3-3 所示:

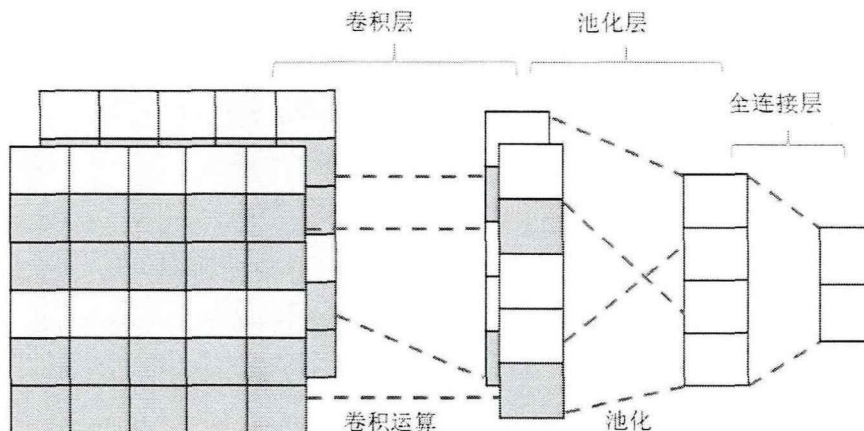


图 3-3 1D-CNN 模型结构

Gated-CNN^[76]结合了门控单元,包括更新门、重置门和输出门。这些门控单元使网络能够调节卷积层的输出,增强其捕获长期依赖关系并识别关键特征的能力。门控机制使 Gated-CNN 能够在选择性地更新和合并新信息的同时保留过去的信息。Gated-CNN 可以看作是在 1D-CNN 的基础上引入门控机制的一种扩展或改进型结构,更加适用于处理包含长程依赖的序列数据。

本文中使用两个 1D-CNN 模型来实现 Gated-CNN 模型,通过将一个使用激活函数的 1D-CNN 模型和一个没有设置激活函数的 1D-CNN 模型的输出结果做乘积处理来实现门控机制。在 Mal-CLAM 模型中:

(1) 本文首先通过批量归一化层^[77]对输入数据进行归一化。批量归一化层通过减去批次平均值并除以批次标准偏差来归一化输入值。它确保特征向量的某些维度不会因此太大而影响训练和数据;

(2) 在数据输入之后应用多个 Gated-CNN 模块来选择数据中的重要和相关信息;

(3) 将多个 Gated-CNN 的输出连接起来,为了避免各个 Gated-CNN 模块的数据维度不一致导致结果失真,再次利用批量归一化层再次进行归一化,并将单个输出连接到 Bi-LSTM 模型的输入。

3.2.3 基于 Bi-LSTM 的依赖关系提取

Mal-CLAM 模型使用 Bi-LSTM^[78]来捕捉 API 序列之间的复杂关系。序列之间的复杂关系。双向长短期记忆网络 (Bi-LSTM) 是 LSTM 的变体,包括两个 LSTM 层,一个用于前向传播 (从左到右处理序列),另一个用于后向传播 (从右到左处理序列)。这意味着它可以同时考虑序列的过去和未来信息,以捕获更多的上下文信息和序列依赖关系。

Bi-LSTM 模型包括前向 LSTM 和反向 LSTM 两部分。在前向 LSTM 中,隐

藏状态是以后向方式顺序计算的，从序列的初始时间步开始计算。相反，反向 LSTM 则以相反的顺序计算隐藏状态，从序列的最后一个时间步开始。这两个 LSTM 的输出会在每个时间步长进行连接或合并，以生成一个综合表示，从而提高捕捉时间数据中的上下文和依赖关系的能力。Bi-LSTM 已在恶意软件检测领域证明了其有效性^[47]。图 3-4 显示了 Bi-LSTM 的架构。

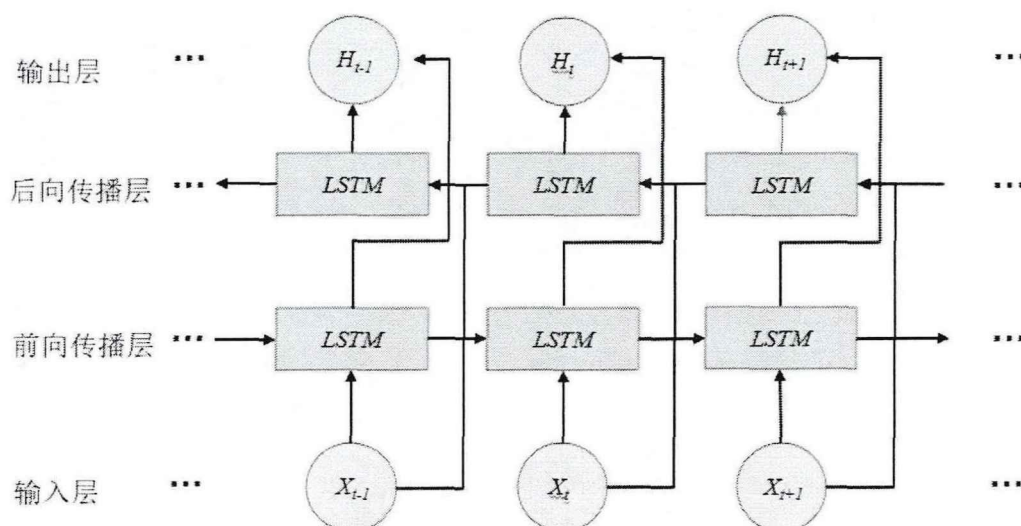


图 3-4 双向长短时记忆网络模型架构

在 Mal-CLAM 模型中，Gated-CNN 的输出被串联起来，并应用一个批量归一化层以减少过拟合。随后，使用一个 Bi-LSTM 模型来捕捉 API 序列中的依赖关系，然后使用全局最大池化层（Global Maximum Pooling Layer）从隐藏向量中提取抽象特征。与普通的池化层不同，全局最大池化层的池化操作是对整个特征图进行的，而不是在局部区域内进行。本模型中全局最大池化层利用在整个序列中观察到的每个信号，将 Bi-LSTM 的输出特征转换为一个固定大小的向量，由于其在一定程度上具有平移不变性，所以特征的位置对池化结果的影响较小，从而能最大限度保留从 API 序列中收集到的关键信息。

3.2.4 基于自编码器的结构特征提取

深度自编码器是一类自编码器神经网络，由多个编码器和解码器层组成，用于学习数据和数据重建的高级表示。训练深度自编码器使编码器能够从输入数据和解码器中学习从这些特征中恢复数据，使它们能够用于特征学习、数据降维、去噪和异常检测等任务的不同应用^[79]。

如图 3-2 (b) 所示，在 Mal-CLAM 模型中深度自编码器模型中使用编码器压缩数据，使用解码器进行数据重构，并添加 Dropout 层防止过拟合。

训练过程分为三步：

- (1) 首先使用编码器将数据压缩到一维，学习数据的深度低维特征；
- (2) 利用解码器对数据进行重构；

(3) 将重构后的数据与初始数据进行计算, 得到重构误差作为序列特征。这能够提高了模型的泛化能力, 减少了人为因素的参与, 加速了样本更新的处理。

自编码器模块具体结构如图 3-5 所示:

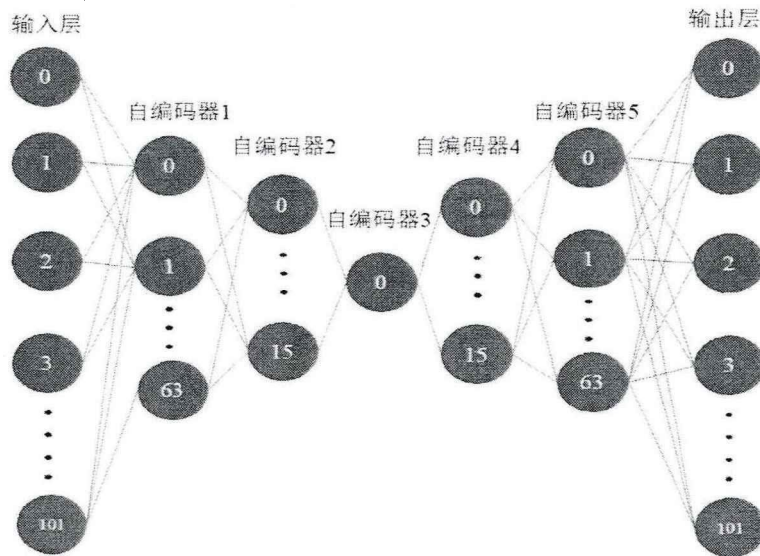


图 3-5 自编码器结构

本文中使用 sigmoid 函数作为自编码器神经元的激活函数, sigmoid 函数是各种实验中使用最广泛的非线性激活函数, 数学形式如 (3-2) 所示:

$$\text{Sigmoid}(z) = \frac{1}{1+e^{-z}} \quad (3-2)$$

该函数把数值进行非线性处理, 将其范围限制在 (0,1) 内, 是一种光滑且严格单调的饱和函数, 易于求导。

Dropout 层^[80]可以使模型在每个训练批次中都会忽略一部分特征 (让该部分的隐藏层节点值变为 0), 这样可以减少隐藏节点间的相互作用, 让模型降低对某些局部特征的依赖。这样做有两个作用: 一是起了取平均值的作用, 它会使某些隐藏层神经元失效, 相当于断开了网络中的联系, 形成了不同的网络构造。各个网络在训练后会有各自的参数, 适应不同的拟合现象, 把这些模型取平均, 对相似的拟合影响不大, 而相反的拟合就可以相互抵消, 从而在整体上降低了过拟合程度; 二是减少神经元之间复杂的共适应关系。神经网络可能会依靠某些特征的固定关系进行判断, 而 Dropout 层能够使神经元随机失效, 这样就可以使有关系的神经元不会次次都出现, 减少参数更新对它们的依赖。这样那些更能表现数据的特征就可以被网络捕获到, 防止了网络对特定特征的过度依赖, 即便该特征丢失, 也能从其他特征中, 学习新的特征, 类似于 L1, L2 正则化, 通过降低权重增强网络对丢失特定神经元的鲁棒性。在具体实现上, 它能够使神经元以概率 p 失去作用, 即让它对应的激活函数的输出变为 0, 概率为 p , 例如本文模型的某隐藏层神经元为 64 个, dropout 概率选为 0.5, 经过 Dropout 层的运作之后, 就会有一半, 即 32 个神经元的值被置为 0。其过程如下:

(1) 首先保持输入输出层的神经元不变, 先使隐藏层的各神经元按设定好的概率使之失效(令其输出值为 0, 但要保存其参数)。

(2) 把数据样本输入网络, 进行前向传播, 然后模型计算得来的损失函数反向传播, 等到这批样本训练完成后, 那些未失效的神经元的参数已经更新完成。

(3) 恢复所有失效的神经元, 这些神经元的参数没有改变, 但之前参与训练的神经元参数已经改变了。然后执行步骤(1)~(2), 不断重复这一过程。

3.2.5 基于多层感知器的恶意软件检测

将 Bi-LSTM 模块和自编码器模块的输出连接起来, 使用 MLP 模型来获得最终的检测结果。MLP (Multilayer Perceptron), 即多层感知机, 是一种人工神经网络模型, 也被称为前馈神经网络 (Feedforward Neural Network)。它是一种基于连接权重的有向图, 包含多个神经元(或称为节点)和多个层次, 基本结构包括接受输入特征的输入层, 位于输入层和输出层之间的隐藏层和给出模型结果的输出层。输出层中每个节点对应一个输出类别或预测目标。MLP 模型广泛用于分类和回归任务。对于复杂的问题, 可以采用深层的 MLP 模型, 即深度神经网络 (Deep Neural Network, DNN)。

本文中 MLP 模块由三个密集层组成, 最后一个密集层输出样本为恶意软件的概率。前两个密集层采用 relu 作为激活函数, 最后一个密集层采用 sigmoid 作为激活函数。relu 函数比较简单, 它输出输入值和 0 之间的最大值, 计算公式如(3-3)所示:

$$\text{Relu}(x) = \max(x, 0) \quad (3-3)$$

relu 函数的输出全部都在正区间, 且运算速率快, 收敛速率也远超 sigmoid 和 tanh。但是这个函数的输出也并非零均值, 而且还可以导致某些神经元永远都无法被激活。

Mal-CLAM 模型在训练时通过对应于每个输入向量的标签进行监督训练。为了最小化训练模型的损失, 二元交叉熵函数作为训练时的损失函数, 具体如公式(3-4)所示:

$$\text{Loss} = -\frac{1}{\text{output size}} \sum_{i=1}^{\text{output size}} y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i)) \quad (3-4)$$

其中, y 是二元标签 0 或者 1, $p(y_i)$ 是输出属于 y 标签的概率。二元交叉熵函数用来评判一个二分类模型预测结果的好坏程度的, 即对于标签 y 为 1 的情况, 如果预测值 $p(y)$ 趋近于 1, 那么损失函数的值应当趋近于 0; 反之, 如果此时预测值 $p(y)$ 趋近于 0, 那么损失函数的值应当非常大。本实验中的恶意软件检测是二元分类任务, 模型的输出是输入样本为恶意软件的概率, 是一维数据。Outputsize 代表模型的输出维度, 实验中设置为 1, 输出为恶意软件的概率。

3.3 实验设置

3.3.1 实验环境

本文的实验环境配置如表 3-1 所示。实验在 Windows11 系统上进行，配置了 GTX 1660Ti 显卡，运行内存 16G，需要 200G 的存储空间。模型基于 Python3.8 开发，使用 Keras、TensorFlow、Sklearn 等框架编写。

表 3-1 实验环境配置表

实验配置	实际设置
系统	Windows11
GPU	GTX 1660Ti
内存	16G
硬盘	200G
编程语言	Python3.8
开发框架	Keras、TensorFlow、Sklearn

3.3.2 实验数据

本文使用 Cuckoo2 沙箱^[81]作为安全虚拟环境运行软件，提取特征。Cuckoo2 沙箱是开源软件，便于本文研究工作的获取和使用，能够根据需要扩展功能，便于应对不同场景；支持多种分析功能，功能强大而且使用简单；此外，当前许多基于 API 序列特征进行恶意软件检测的研究工作都是使用的 Cuckoo2 沙箱，便于实验结果的对比和研究成果的复现。

本文使用的数据集中，所有的软件均为 Windows 系统下的可执行文件。恶意软件来自 VirusShare 网站，良性软件来自免费软件分享网站，包括 Softonic.com、Sourceforge.net 和 Portableapps.com。恶意软件包括病毒、蠕虫、木马、勒索软件等，良性软件包括几乎所有应用软件类别，包括系统、互联网、游戏、娱乐、媒体软件等。通过开源的软件收集软件进行收集，而后通过在 Cuckoo2 沙箱中运行软件样本提取 API 调用序列，预处理后形成本文用于模型训练的数据集。本文使用的数据集包含 15000 个样本，训练集和测试集各包含 7500 个样本，每个样本包括了不重复的 1000 个 API 调用。每个 API 调用特征分为三部分：API 名称、API 类型和 API 使用参数，共 102 维。

具体的数据处理过程如下所示，包括 API 特征的内容、同义函数的处理、特征向量转换和 API 序列精简。

(1) 丰富 API 特征包含内容：很多利用 API 调用序列进行检测的恶意软件检测算法在选取具体的 API 特征时通常只考虑了 API 名称，然而同样的 API 调用函数，执行的对象不同，其性质是完全不同的，如系统信息获取类型的 API 调

用函数，其敏感程度相对于文件和注册表操作类型的 API 调用函数要低很多；此外，API 调用的参数，如操作文件大小，频率，操作文件类型等对判断软件行为特征也十分重要。因此，本文使用的 API 序列中包含了 API 名称、操作类型、参数等特征，为判断软件行为特征提供更全面的依据。

(2) 消除同义 API 函数的影响：Windows 平台上存在一些名称不同但功能相同的 API 函数，如 LoadLibraryA 和 LoadLibraryW，都是库加载函数，区别在只于编码不同，因此在处理时将其视为同一个函数，删除其后缀在进行后续处理。

(3) 将 API 特征转换为固定长度向量：每个 API 名称由多个单词组成，每个单词大写的第一个字母，例如“RegCreateKey”。处理时将 API 名称拆分为单词，然后通过哈希映射将其转换成索引，这样既可以减轻数据大小，又方便去重操作。至于 API 函数的分类，依照官方文档查找对应类别即可。API 函数的参数分为两类，数值型和字符串型。数值型没有实际意义，但一旦代表一类具体的资源，如数字“22”没有实际意义，但其可能代表着 API 调用的端口号、传送数据的大小，这种情况下使用参数名称；字符串型参数意义相对丰富，根据具体类型进行分类，分为资源路径、IP 地址、URL 资源定位符、DLL 链接库、注册表等，通过哈希方法将其映射为向量特征。

(4) 精简 API 序列：许多恶意软件为了躲避检测，通常采取的方法是在攻击行为中插入大量的正常行为，用以干扰分析，这导致提取的 API 序列会包含多个连续相同的 API 或对应的片段，使得 API 序列包含的有效信息过少，如果使用更长的 API 序列则会导致数据集过大，大大降低检测效率，影响干扰对恶意软件行为的分析检测，增加了训练时间。因此，在数据处理时，对 API 序列进行了去重操作，以让其能够有效的反应软件的行为特征。

经过上述所有处理后每个软件的特征变成了一个包含 1000 个 API 调用特征的序列，每个 API 调用被转化成了固定长度为 102 的向量值，其中名称占 8 位、类别占 4 位、参数占 90 位。

本文使用的数据集信息如表 3-2 所示，在数据集中，标签为 0 代表良性软件，标签为 1 代表恶意软件。

表 3-2 恶意软件检测数据集

数据集	良性软件	恶意软件	总计
训练集	4785	2715	7500
测试集	2808	4692	7500

该数据集包含 15000 个样本，其中训练集 7500 个样本，包括 4785 个良性软件，2715 个恶意软件；测试集 7500 个样本呢，包括 2808 个良性软件，4692 个恶意软件。每个样本包括了不重复的 1000 个 API。为了更好的描述软件运行时

的信息特征，每个 API 信息分为三部分：API 名称、API 类型和 API 使用参数，共 102 维。

为了测试模型的泛化能力，本文将数据集拆分为训练集和测试集，两个数据集包含的测试样本不重复，以保证使用训练集数据训练后的模型检测的都是未接触过的软件，以便充分验证模型的检测准确率和鲁棒性。

3.3.3 评价指标

为了准确评估模型的检测效果，本文使用准确度、召回率、F1-score 和 AUC 值来评估恶意软件检测模型的检测性能并进行比较。它们是从混淆矩阵^[82]中而来的，也称为误差矩阵数据，实际上就是一张展示预测结果正常异常与否的表。在这张表中数据被分为四类：TP (True Positive) 表示将正例预测为正例，TN (True Negative) 表示将负例预测为负例，FP (False Positive) 表示将负例预测为正例，FN (False Negative) 表示将正例预测为负例。

准确率 (ACC)，是一种常用于评估分类模型性能的性能指标，尤其是在多类别分类问题中，它表示正确分类的样本数量与样本总数之间的比率。准确率的值介于 0 和 1 之间，其中 1 表示模型的所有预测都是正确的，0 表示所有预测都是错误的。高准确率通常是一个很好的性能指标，但在某些情况下，如不平衡数据集，准确率可能不足以全面评估模型的性能。例如，在互联网广告中，点击次数非常少，通常只有千分之几，如果使用 ACC，即使所有预测都进入负类别（无点击）ACC 也超过 0.99，这就没有意义了。准确率的计算方法如 (3-5) 所示：

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} \quad (3-5)$$

召回率 (Recall) 又称真阳性率或灵敏度，是用于衡量二元或多类别问题中模型性能的最重要指标之一。召回率表示模型成功检测到的阳性类别样本的比例，即所有实际阳性类别样本中有多少被模型正确识别。召回率的计算方法如 (3-6) 所示：

$$\text{Recall} = \frac{TP}{TP+FN} \quad (3-6)$$

F1 值结合了精确度和召回率，提供了一个平衡的指标。F1 值通常用于处理不平衡的数据集，或在精确度和召回率之间进行权衡。F1 分数的计算方法如 (3-7) 所示：

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3-7)$$

AUC (曲线下面积) 是 ROC 曲线下的面积，其数值介于 0.1 和 1 之间。AUC 可以直观地评估分类器的性能，值越大越好。AUC 值也是一种概率数值，当随机地选取正样本和负样本时，当前基于分数计算的分类方法将是在负数据前方的正数据。AUC 值越高，按照当前排序方法对负样本前面的正样本进行排序的概

率也更大，因此可以更好的对它们进行排序。AUC 值的计算方式相对复杂，有各种各样的计算公式，目前相对比较简单易懂的 AUC 值的计算方法如（3-8）所示：

$$AUC = \frac{\sum_{i \in \text{positiveClass}} \text{rank}_i - \frac{M(1+M)}{2}}{M \times N}$$

(3-8)

其中，M 为正类样本的数目，N 为负类样本的数目，每个测试样本属于正样本的概率为 Score，对 Score 从大到小排序，然后令最大 Score 对应的样本的 rank 为 N，第二大 Score 对应样本的 rank 为 N-1，以此类推。

3.4 消融实验

本文所提出的 Mal-CLAM 模型包含多个可以灵活组合的模块，例如 Gated-CNN、Bi-LSTM 和自编码器等。为了研究各个模块的起到的作用，增强模型的可解释性，本文通过保持其他结构不变并仅更改测试模块来进行多组比较实验。这些实验用于研究不同模块对整体模型性能的影响，分析它们起到的作用以及原因，并作为本文最终模型结构优化的基础。

3.4.1 门控卷积消融实验

本文设置了四组实验：使用内核大小为 2 的 Gated-CNN；两个内核大小为 2、3 的 Gated-CNN；三个内核大小为 2、3 和 4（Mal-CLAM 模型）的 Gated-CNN；四个内核大小为 2、3、4 和 5 的 Gated-CNN。实验结果如图 4-5 所示，绿色方框代表本文提出的模型的性能表现，可以看到 Mal-CLAM 模型在 ACC、F1-score、Recall 和 AUC 值四个方面表现都是最好的，证明了本文使用的构造的优越性。

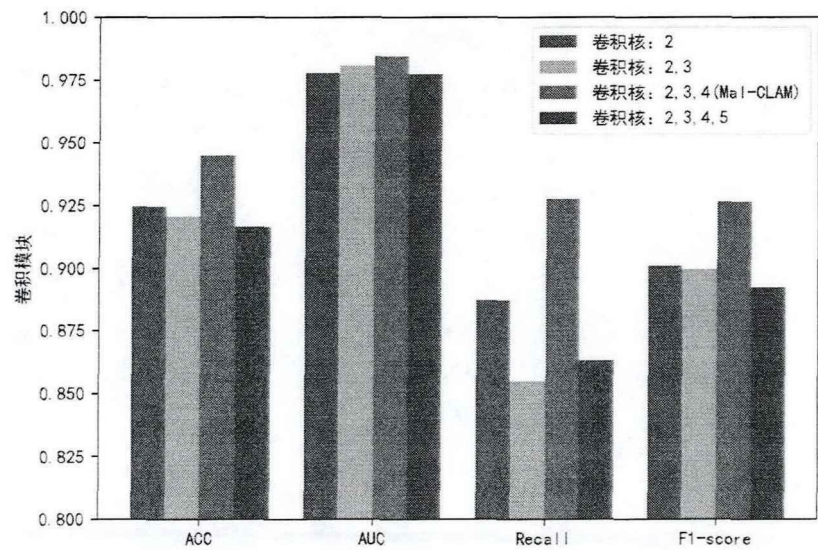


图 3-6 卷积模块消融实验结果对比

图 3-6 描述了恶意软件检测模型在设置不同数量的 Gated-CNN 情况下的性能比较。可以看出，增加 Gated-CNN 的数量并不一定会带来性能提升。使用一

个 Gated-CNN 的性能甚至高于两个 Gated-CNN，以及四个 Gated-CNN。三个 Gated-CNN（内核大小 2,3,4）的模型在所有四个评估指标中都高于其他模型。因此，在本文的模型中使用该配置。

在模型中设置多个门控卷积模块的目的是从多方面、多角度去提取 API 序列中的有效信息。传统的卷积神经网络存在一个问题就是将所有元素都视为有效元素，导致提取过多无效信息干扰检测结果，门控卷积网络虽然在一定程度上解决了该问题，但是实验中如果设置过多，会导致无效元素也被视为有效特征参与到最终检测过程，而干扰了实验结果。

3.4.2 Bi-LSTM 消融实验

本文设置了三组实验，即不使用 Bi-LSTM 的模型，使用一个 Bi-LSTM（Mal-CLAM 模型）的模型，堆叠了两个 Bi-LSTM 的模型。三组实验的结果对比如图 3-7 所示，橘色方框代表本文提出模型的检测性能表现，可以看到 Mal-CLAM 模型在 ACC、AUC 值和 F1 值三个指标上表现都最好，综合表现最佳。

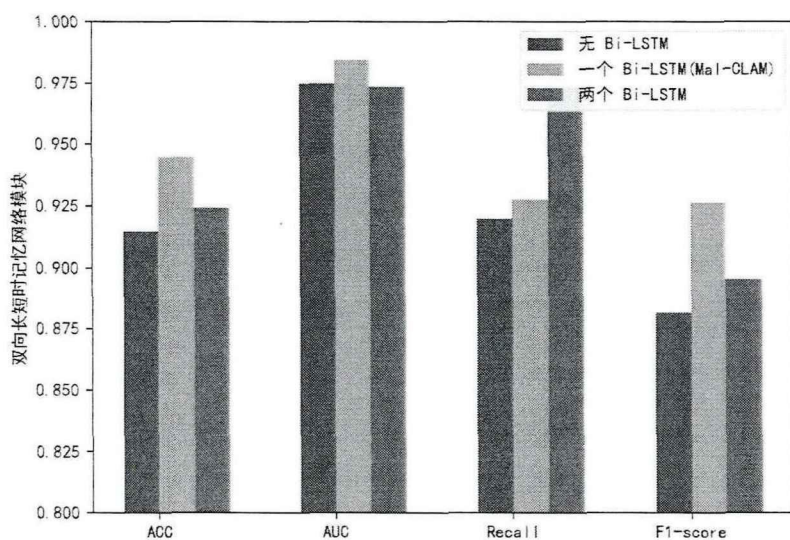


图 3-7 Bi-LSTM 模块消融实验结果对比

通过分析实验结果可以得知，没有使用 Bi-LSTM 模型的模型展示的检测性能在所有指标上都低于其他两个模型，这表明 Bi-LSTM 在对 API 序列的分析中很重要。使用了一个 Bi-LSTM 的模型在除 Recall 之外的其它指标上都具有最佳性能，考虑到使用两个 Bi-LSTM 模块训练模型消耗的时间比仅使用一个 Bi-LSTM 模型的模型训练消耗的多将近一倍。因此，综合考虑下来本文最终选择了使用一个 Bi-LSTM 模块来搭建检测模型。

通过无 Bi-LSTM 模型和使用一个 Bi-LSTM 模型的实验结果对比可以看出，API 调用序列中 API 之间的依赖关系对检测模型学习恶意软件的行为特征，提高检测准确率十分重要。由于实验为了提升检测效率，所有 API 序列的长度都是 1000，使用一个 Bi-LSTM 模型已经足够获取序列中行为特征信息了，因此再增

加 Bi-LSTM 模型也不会在检测性能上有所提升。当面对更加复杂的恶意软件攻击行为时,改变 API 序列的长度,探究不同长度的 API 序列对检测结果的影响,以及不同 Bi-LSTM 模型检测效果与 API 序列长度是否相关具有重要的研究价值。

3.4.3 自编码器消融实验

本文设置了四组实验,没有使用自编码器的模型,使用由两个密集层组成的自编码器的模型,使用由三个密集层(Mal-CLAM 模型)组成的自编码器的模型,使用由四个密集层组成的自编码器的模型。实验结果如图 3-8 所示,绿色方框代表的本文提出模型性能表现,可以看到 Mal-CLAM 模型在 ACC、F1-score、Recall 和 AUC 值四个方面表现都是最好的,说明本文采取的架构合理。

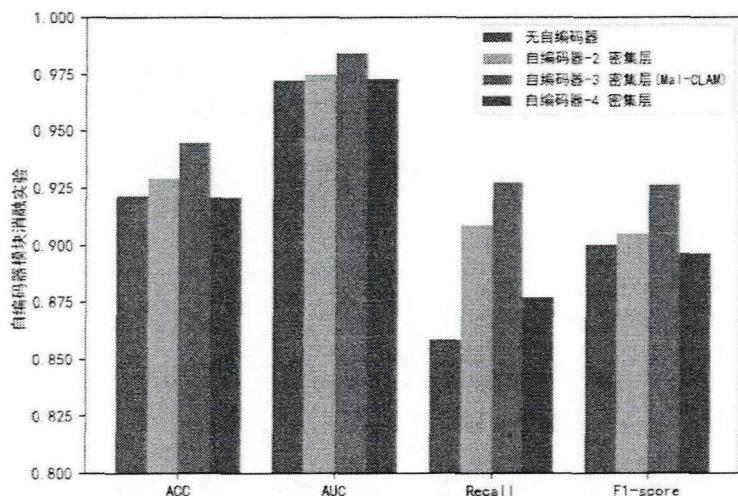


图 3-8 自编码器模块消融实验结果对比

根据实验结果可知,在没有包含自编码器的情况下,检测模型的性能明显较差,这进一步证明通过自编码器从 API 序列中提取深层特征对提高恶意软件检测模型性能的重要性。在这种情况下,密集层的数量是一个重要的参数。随着自编码器模块的密集层数量的增加,模型的复杂性也随之增加,可能导致过度拟合现象和检测性能的下降。在此次消融实验中,使用四个密集层后,检测模型的性能出现大幅下降,说明自编码器模块并不是使用越多密集层就能取得更好的效果。因此,本文根据实验的结论选择了使用三个密集层的自编码器来构建检测模型。

3.5 实验结果及分析

3.5.1 恶意软件检测模型性能评估

本文通过消融实验研究了 Mal-CLAM 模型各个模块的作用,并据此确定了最优模型结构:

在卷积模块中,使用三个卷积核大小为 2、3 和 4 的 Gated-CNN。所有卷积层的滤波器大小为 128,步幅为 1。

在 Bi-LSTM 模块中, 每个 LSTM 的单元数为 100。

在深度自编码器模块中, 编码器包括三个输入维度为 64、16 和 1 的自编码层; 解码器包括三个输入维度为 16、64 和 102 的自编码层, 输出重构样本; 最后使用 Subtract 层计算重构数据和原始数据之间的重构误差。所有自编码层都由 sigmoid 函数激活。为了尽可能避免过度拟合现象的发生, 本文在每个密集层之后使用丢弃概率为 0.5 的 Dropout 层。

在 MLP 模块中, 本文设置了三个具有 128、64 和 1 个单元的密集层, 每个层都使用 relu 函数激活。然后使用丢弃概率为 0.5 的 Dropout 层来减少过度拟合。本文使用 sigmoid 函数在密集层之后输出检测概率。

每个模型都训练了四次, 迭代次数为 25, 最后取每次训练结果的平均值。

为了证明所提出模型的性能, 本文选择了其它的恶意软件检测模型进行比较:

模型一: Xing 等人^[40]提出一种基于自编码器重构误差检测模型, 该模型提取软件的静态特征和动态特征, 利用该特征构建恶意软件的灰度图像, 并与深度学习模型中的自编码器相结合, 利用了恶意软件的低维特征实现恶意软件检测。

模型二: Agrawal 等人^[47]提出了一种基于动态分析的两阶段模型, 该模型以软件运行时的 API 序列为样本, 加入了对 API 调用参数的分析, 使用长短时记忆网络学习样本特征, 使用多层感知器对软件进行检测。

模型三: Catak 等人^[64]提出了一种 Windows 平台上利用双层长短时记忆网络的恶意软件检测模型, 该模型获取每个软件运行的 API 调用序列, 先后输入到两个不同的 LSTM 模型中以获取软件的特征, 然后将特征输入到 tanh、relu、sigmoid、softplus、softsign、softmax 和 linear 这七个激活函数中, 通过这些函数的输出投票判定软件的性质。

模型四: Zhang 等人^[61]聚焦在 API 参数对恶意软件检测的影响, 通过哈希技术对 API 调用相关的函数进行编码, 通过结合卷积神经网络和长短时记忆网络提取 API 序列的特征, 对恶意软件进行检测。

经过实验后, 各个模型的具体检测指标如表 3-3 和图 3-9 所示:

表 3-3 模型实验结果

模型	ACC	Recall	F1-score	AUC
Xing ^[40]	0.8666	0.8289	0.8199	0.9344
Agrawal ^[47]	0.9305	0.9047	0.9074	0.9797
Catak ^[64]	0.9292	0.8976	0.9063	0.9774
Zhang ^[63]	0.9352	0.8954	0.9153	0.9833
Mal-CLAM	0.9449	0.9275	0.9263	0.9843

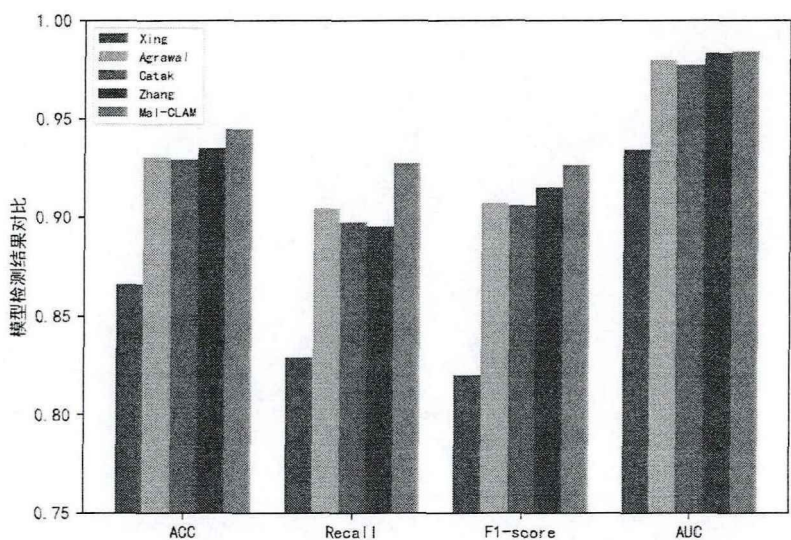


图 3-9 恶意软件检测模型结果对比

图 3-9 中，紫色条框代表本文所提出的 Mal-CLAM 模型，检测结果达到了 94.49% 的准确率、92.75% 的召回率、92.63% 的 F1 值和 98.43% 的 AUC 值，从图中可以看到 Mal-CLAM 模型在四项指标上都高于其它模型。

Mal-CLAM 模型使用的特征包括了 API 名称、类型、使用参数等，内容特征包含的信息更丰富；其次，利用 API 序列中上下文依赖关系和结构特征的组合进行恶意软件检测，在实现时通过卷积网络捕获 API 序列的依赖关系和关键特征，降低了无效特征的干扰，再利用双向长短时记忆网络从前向和后向两个方向上进一步提取 API 序列之间的上下文依赖关系，同时利用自编码器对 API 序列压缩重构，获取结构特征，使得模型用于检测恶意软件的特征更全面；此外，本文还通过消融实验研究了各个模块起到的作用并获得了模型的最佳设置方案。因此 Mal-CLAM 模型在恶意软件检测的各项指标上的表现高于其它的对比模型。

在训练性能上，Mal-CLAM 模型训练所需要的时间平均为七个小时，低于模型二至模型四所需要的时间，模型一由于仅使用自编码器模型，结构较为简单训练时间较短。可以证明，Mal-CLAM 模型在训练时间和检测效果均表现良好。

3.5.2 恶意软件检测模型抗干扰能力评估

上述实验过程中使用的数据集都是准确标注恶意软件和良性软件样本的。然而，恶意软件迭代的速率日益加快，导致各种新的、当前未知的恶意软件的快速出现。当前认为没有安全风险的普通软件可能是一种新颖的、未被识别的、可以避开当前使用的检测方法的恶意软件。因此，不可避免会导致一些新形式的恶意软件暂时逃脱检测，伪装成良性软件并渗透到恶意软件检测模型的数据集中的情况出现。这种对数据集的污染最终会影响恶意软件检测模型的性能。为了直观地评估这种情况对模型的影响，本文将 5% 的恶意软件数据集指定为良性软件，通过实验来探究各个模型对这种数据集污染情况的抗干扰能

力。实验过程和配置保持不变，结果如图 3-10 所示：

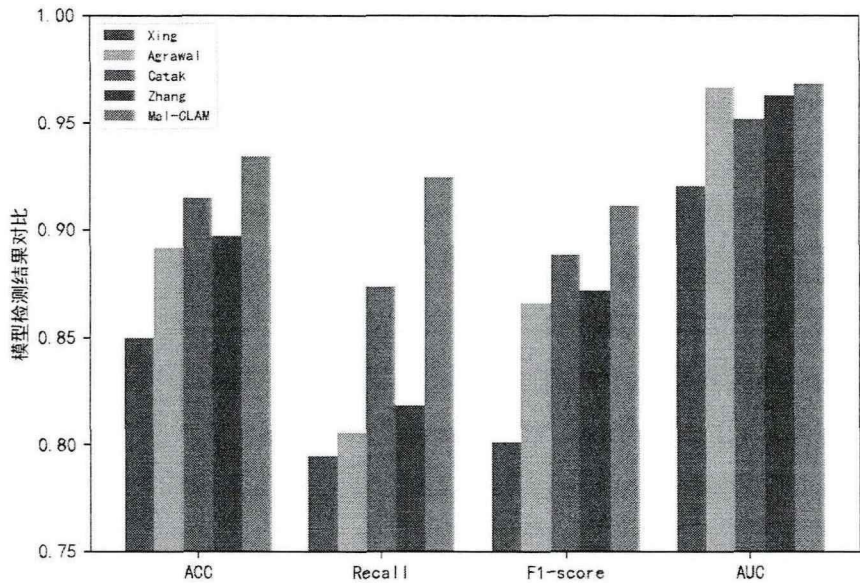


图 3-10 干扰下恶意软件检测模型结果对比

各个模型的具体检测指标如表 3-4 所示：

表 3-4 受干扰下的模型实验结果

模型	ACC	Recall	F1-score	AUC
Xing ^[63]	0.8499	0.7946	0.8011	0.9205
Agrawal ^[63]	0.8917	0.8055	0.8662	0.9665
Catak ^[63]	0.9150	0.8738	0.8884	0.9516
Zhang ^[63]	0.8973	0.8181	0.8718	0.9627
Mal-CLAM	0.9344	0.9245	0.9111	0.9681

由图 3-8 可知，受数据集干扰的影响后，各个模型的检测效果均出现不同程度的降低，但本文所提出的 Mal-CLAM 模型仍达到了 93.44%的准确率、92.45%的召回率、91.11%的 F1 值和 96.81%的 AUC 值，各项性能指标仍然高于其他模型。在模型检测准确率上，Mal-CLAM 模型仅下降了 1.05%，下降幅度低于其它模型的 1.67%、3.88%、1.42%、4.79%；在模型检测 F1 值上，Mal-CLAM 模型仅下降 1.53%，下降幅度低于其它模型的 1.88%、4.12%、1.99%、4.35%；在模型检测召回率上，Mal-CLAM 模型仅下降 0.3%，下降幅度远低于其它模型的 3.49%、9.88%、2.38%、7.73%；而在模型检测 AUC 值上，Mal-CLAM 模型下降 1.62%、高于 Xing 的模型 1.29%和 Agrawal 的模型的 1.32%，低于其它模型的 2.58%、2.06%，该项指标整体受数据干扰影响程度相差不大，表现尚可。

综上所述，可以看出 Mal-CLAM 模型在受到干扰后 ACC、F1-score 和 Recall 三个指标相比其它检测模型受到的影响最小，AUC 值方面的表现出的抗干扰能力也较好，可以证明 Mal-CLAM 模型比其他检测模型具有更好的抗干扰能力，

具备更强的鲁棒性。

Mal-CLAM 模型使用 API 序列的上下文依赖关系和结构特征检测，使用的信息更丰富，并且卷积模块处理后排除一些无效的信息和干扰信息，此外，在自编码器模型中加入了 Dropout 层，通过减少神经元之间复杂的共适应关系，迫使网络去学习更加泛化的特征，因此模型整体能表现出更强的抗干扰能力，具有更强的鲁棒性。当前新型恶意软件攻击行为、隐藏方式越来越多样化、隐蔽化，在收集训练数据时难免会有尚未被检测出的恶意软件隐藏其中，而深度学习算法又十分依赖数据的准确性，具备高抗干扰能力、强鲁棒性对于恶意软件检测算法意义非凡。

3.6 本章小结

本章介绍了基于 API 序列特征和依赖关系的深度学习恶意软件检测模型的整个工作流程，并对其中的关键技术和算法进行了阐述。首先是数据的收集和处理阶段，先从网络上收集软件样本放置到安全虚拟环境中运行获取运行日志，并从中获取 API 调用序列，然后采用去重操作去除重复的 API 调用；然后对模型使用的检测算法进行了详细的介绍，包括了整体架构，卷积模块、Bi-LSTM 模块、自编码器模块、MLP 模块；通过消融实验研究了各个模块起到的作用，并确定模型的最佳结构设置；实验结果表明本文模型的检测准确率达到了 94.49%，相比于其它恶意软件检测算法在检测性能上表现更佳，并且具备更强的鲁棒性。

第四章 基于深度学习的恶意软件检测系统设计与实现

本章节针对恶意软件检测系统准确率低和模型更新慢等问题,通过需求分析、系统设计,将本文的恶意软件检测模型研究成果落地,基于 Mal-CLAM 模型以及其它检测模型搭建了恶意软件检测系统,并设计测试用例,完成对系统功能的测试与验证,开发对应的可视化界面。

4.1 系统需求分析与设计

4.1.1 系统需求分析

随着网络的快速普及,网络安全问题频频出现,给个人、企业甚至是国家带来巨大经济损失。当前对安全产品的需求越来越大,对着技术的发展,恶意软件的更新速度、攻击方式层出不穷,传统的恶意软件检测技术已经不足以应对当前的网络形势,因此需要更为有效的防护技术。本文提出的恶意软件检测模型具有准确率高、泛化性强等优点,能够高效准确地检测恶意软件,提升网络安全防护能力

通过对已有恶意软件检测系统的功能分析以及用户对于恶意软件检测的需求分析,以本文提出的基于深度学习的恶意软件检测模型为核心,设计并实现一个功能齐备的恶意软件检测系统。该系统对外提供以下功能:

(1) 登录注册: 用户需要注册账号并登录以使用系统。并根据每个用户的实际情况赋予不同的权限。

(2) 恶意软件检测: 恶意软件检测包括两个子功能,一是日志提取功能,接收用户上传的软件,放入到 cuckoo2 沙箱中运行,获取软件运行时的日志信息;二是检测功能,首先从日志中提取检测特征,然后通过检测模型获取检测结果,并返回给用户。同时,系统还会将恶意软件的特征以及检测结果存储起来,便于后续的查询以及模型的更新。

(3) 检测模型更新: 由于恶意软件会随着时间的推移不断更新,因此检测模型也需要随之更新,以更好地应对最新的恶意软件。该功能获取最新的软件用来更新检测模型,使系统能够获取准确检测最新软件的能力。

(4) 历史记录搜索: 用户可以使用软件的名称、散列值等信息查询系统中相应软件的检测信息。用户可以依据软件的名称、哈希值等信息进行查询,避免重复检测浪费资源。

(5) 检测结果可视化: 系统会将近期检测结果保存到数据库,并按照时间顺序展示出供用户查看,以使用户对近期的安全形势和恶意软件有初步了解。

此外,恶意软件检测系统在提供服务时,为方便用户使用,系统的长期运营维护还需要具备以下特点:

(1) 用户友好性:恶意软件检测系统需要满足普通用户对于恶意软件检测的需求,他们通常不具备恶意软件检测相关知识和技能,因此检测系统的使用界面要清晰明了,能够快速上手,用户能够低成本学会系统功能的使用。

(2) 安全性:恶意软件检测系统涉及可执行的恶意软件,其中有一些恶意软件可以自动进行攻击,因此要妥善存储恶意软件样本,进行无害化处理,避免被检测的恶意软件攻击。

(3) 扩展性:检测系统当前只针对可执行文件进行检测,随着时间推移需要增加对其它类型文件的检测,以及新的功能,因此系统的架构要能够快速安全地添加新功能。

(4) 可移植性:由于用户使用的操作系统不同,硬件不同,部署环境不同,为满足不同的场景需求,系统还应该具备可以执行,能够在不同操作系统,不同硬件条件下正常部署使用。

4.1.2 系统架构设计

系统的运行逻辑如下:恶意软件检测系统运行时,用户首先需要注册账号并登录才能够使用各项功能;使用检测功能时,系统首先利用沙箱获取软件的运行日志并提取特征,然后将该软件的特征存储到数据库,同时将特征输入到检测模型获取检测结果返回给前端展示;当检测的新型软件到达一定数量后,系统就会构造新的数据集训练新的模型;用户使用历史记录搜索功能时输入软件信息,如名称、sha256 值等,系统从数据库搜索结果并返回给用户;用户使用可视化模块时,系统根据时间顺序和用户所在页面页码从数据库批量获取检测记录返回给用户。

针对上述使用场景,恶意软件检测系统通过浏览器的 Web 交互模式作为可视化展示,使用 FastApi 框架开发。开发时遵循 MVC 架构,将系统分为三层:

模型层 (Model): 模型层将承载系统内的数据和实现业务功能。数据承载包括对实体类的数据的定义使用;业务功能包括对用户请求的响应,比如进行恶意软件检测。

视图层 (View): 视图层负责界面设计与展示,提供给用户可视化操作,将请求发送给控制器层进行处理。

控制器层 (Controller): 控制器层负责转发请求,对请求进行处理,根据用户不同请求,交给不同的模块处理,并将结果返回。

本文的恶意软件检测系统架构遵循上述 MVC 架构,分为以下三层,如图 4-1 所示:

视图层：视图层是信息展示和与用户交互的接口，负责接收用户的请求并展示最终的结果。

应用层：负责处理用户的请求，转发到相应的功能模块，并将处理结果返回给用户。

数据层：数据层负责定义数据实体结构，并与数据库进行交互。数据层存储的信息包括：检测样本信息、检测结果、用户信息、检测记录等。数据库使用MySQL实现，通过 SQLAlchemy 模型建立系统于数据库之间的映射关系，具体的数据库表设计见 4.2 恶意软件检测系统实现章节各模块的实现。

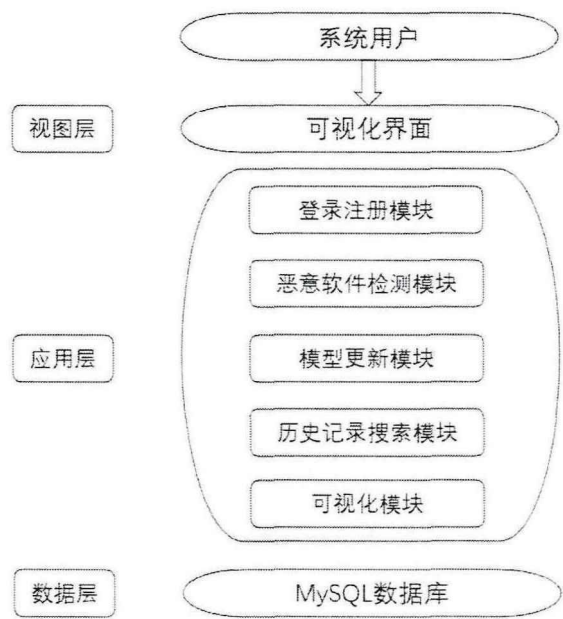


图 4-1 恶意软件检测系统架构图

4.1.3 系统模块设计

本文设计的系统包含登录注册模块、恶意软件检测模块、模型更新模块、历史记录搜索模块、可视化模块以及数据库存储功能，如图 4-2 所示：

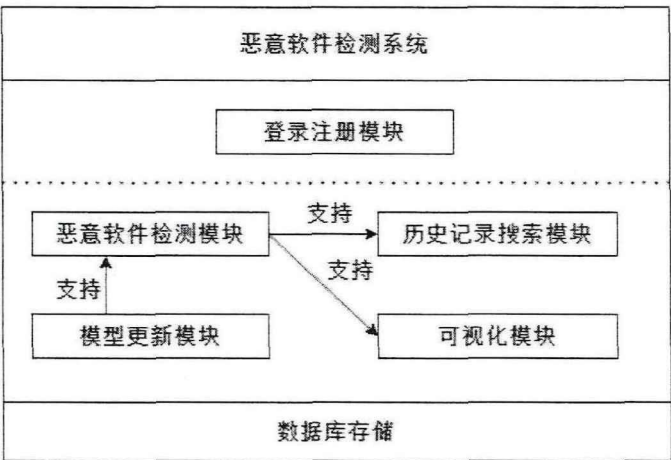


图 4-2 恶意软件检测系统功能

登录注册模块负责对系统的功能使用进行权限控制；恶意软件检测模块依靠模型更新模块获取最新的检测模型，同时给出检测结果支持历史记录搜索模块和可视化模块。

（1）登录注册模块

用户需要注册账号，已保存个人的使用信息。需输入用户名、密码与确认密码，当用户名未注册且确认密码与密码相同时，用户注册成功；用户输入用户名和密码，当密码输入正确后登录成功。未登录时用户不能使用系统其他功能。

（2）恶意软件检测模块

恶意软件检测模块包含特征提取和恶意软件检测两部分。日志提取部分由 Cuckoo2 沙箱和特征提取功能构成，将用户上传的可执行文件放置到 Cuckoo2 沙箱中运行，获取 JSON 格式的日志；恶意软件检测部分首先从运行日志中 API 调用序列作为检测特征，再调用检测模型进行检测，并将检测信息返回给用户。系统使用了包括本文提出的 Mal-CLAM 模型在内的五个恶意软件检测模型，给出各自结果以供用户判断。

（3）检测模型更新模块

随着时间推移会不断有新的恶意软件出现，并具备新的攻击方式和特征，因此恶意软件检测模型也要不断地利用新的恶意软件训练更新，否则就会导致检测的准确率下降，从而不能满足用户需求。本文设计的恶意软件检测系统会保存用户上传检测的软件样本特征，并利用这些软件特征形成新的数据集训练更新模型。

（4）历史记录搜索模块

历史记录搜索功能，用户可以使用软件的名称、md5 值、sha256 值、sha1 值等软件样本不变的散列特征去查询对应软件的检测信息。用户在检测前可以先使用搜索功能搜索软件的检测信息，如果系统已经对该软件进行检测则可以直接返回结果，无需再进行检测。

（5）可视化模块

可视化功能，检测系统会将所有用户的检测结果以及在检测模型更新时收集检测的软件保存下来，按照时间顺序展示出供用户查看，由于记录可能很多，因此提供了分页功能。可视化模块可以让用户了解近期流行的恶意软件，增强防范意识。

4.2 恶意软件检测系统实现

本文设计的恶意软件检测系统前端使用的是 Vue3 和 ElementUI 编写；后端使用 Python 语言编写，FastApi 框架开发；数据库使用 MySQL 实现。系统整体采用前后端分离架构实现，便于开发和后续功能的扩展。

4.2.1 登录注册模块实现

登录注册的运行流程如图 4-3 所示，用户需要注册账号，输入用户名，如果用户名重复则需要重新输入；然后输入密码并确认密码，若两次输入密码不一致则需要重新确认输入的密码；最后输入用户名密码登录，如果不正确则重新登录。用户的登录状态使用 JWT 技术保存，用户登录时会将用户信息存储到 JWT token 中，跳转到其他页面时需要传递 token，如果没有 token 则无法登录到对应页面使用功能。

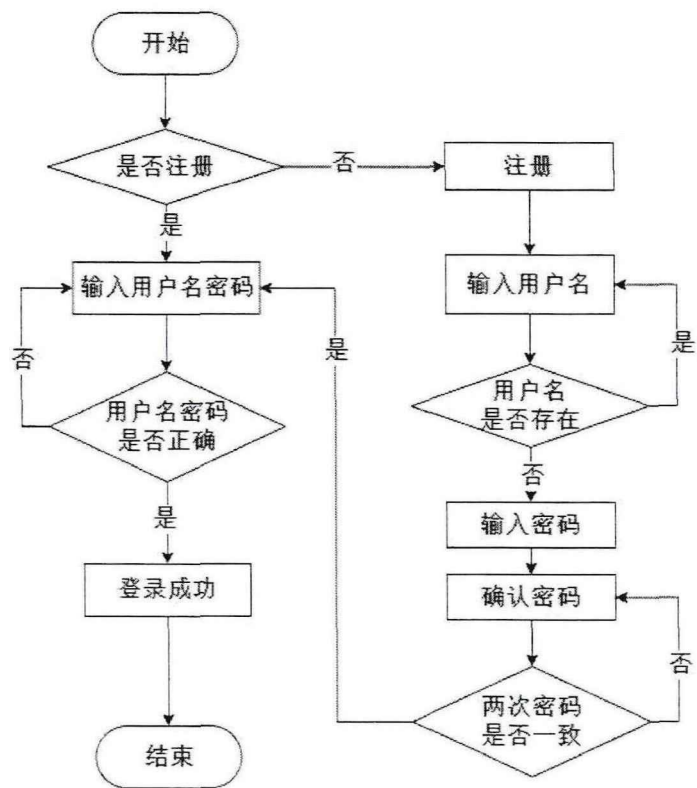


图 4-3 恶意软件检测系统登录注册模块

用户的账户信息存储在用户表中，为了保证数据安全用户密码在存储时使用了 md5 进行处理。用户表包含信息如表 4-1 所示：

表 4-1 用户表描述

字段名	数据类型	描述
id	Int	用户 id
username	Varchar	用户名
passwrod	Varchar	用户密码
time	Datetime	用户注册时间

4.2.2 恶意软件检测模块实现

恶意软件检测运行流程如图 4-4 所示，系统接收用户上传的软件样本，上传到 cuckoo2 安全沙箱中运行，获取软件运行日志，从日志中提取 API 调用序列并

进行特征处理：在执行检测前会调用搜索功能，查看是否已经有对应软件的检测记录，如有直接返回给用户对应结果；如果没有对应搜索记录，将获取的特征存储起来，然后输入到恶意软件检测模型中进行检测，为了支持后续模型的快速更新，获取的检测结果会存储到数据库中，最后返回给用户。

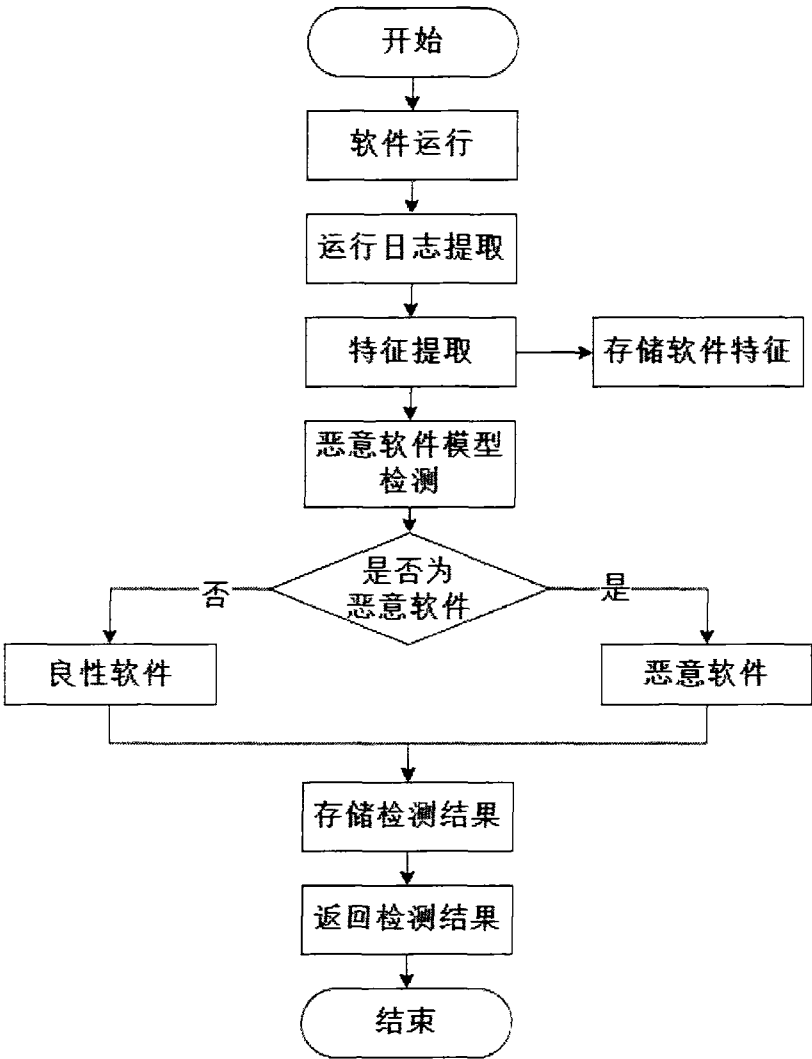


图 4-4 恶意软件检测系统检测模块

软件样本表存储软件的名称、软件特征等信息，具体如表 4-2 所示：

表 4-2 软件样本表描述

字段名	数据类型	描述
id	Int	软件 id
name	Varchar	软件名称
feature	Varchar	软件特征
time	Datetime	软件存储时间

检测信息表则存储了软件检测的返回结果，包括名称、散列值、检测结果等，具体信息如表 4-3 所示：

表 4-3 检测信息表描述

字段名	数据类型	描述
id	Int	检测记录 id
user_id	Int	用户 id
filename	Varchar	文件名称
sha1	Varchar	文件的 sha1 哈希值
sha256	Varchar	文件的 sha256 哈希值
md5	Varchar	文件信息摘要
size	Int	文件大小
type	Varchar	文件类型，本实验中都为.exe 类型
time	Datetime	软件检测时间
model1	Tinyint	恶意软件检测模型：Mal-CLAM 模型
model2	Tinyint	恶意软件检测模型：来自 Xing 等人的研究
model3	Tinyint	恶意软件检测模型：来自 Agrawal 等人的研究
model4	Tinyint	恶意软件检测模型：来自 Catak 等人的研究
model5	Tinyint	恶意软件检测模型：来自 Zhang 等人的研究
result	Int	统计检测模型检测的结果，0-1 表示低风险；2-3 表示中等风险；4-5 表示高风险

4.2.3 模型更新模块实现

模型更新模块运行流程如图 4-5 所示，更新时利用数据库中新增的检测的软件样本和从软件样本库中随机抽取原有的数据样本，构成新的训练数据集，包括最新的恶意软件和良性软件，开始训练新模型，训练完成后存储起来，系统使用的模型替换为新的恶意软件检测模型。同时将模型的更新信息向用户进行了展示。

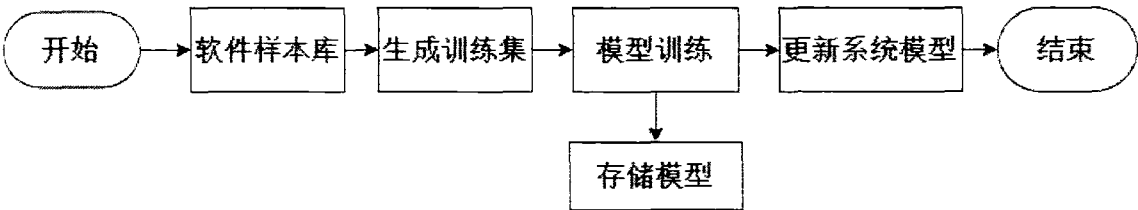


图 4-5 恶意软件检测系统模型更新模块

4.2.4 历史记录搜索模块实现

历史记录搜索模块运行流程如图 4-6 所示，用户输入可以使用软件名称、sha1、sha256、md5 等哈希值搜索软件历史检测记录，如果没有搜索到相应结果系统返回空；如果搜索到结果、系统会返回检测记录。

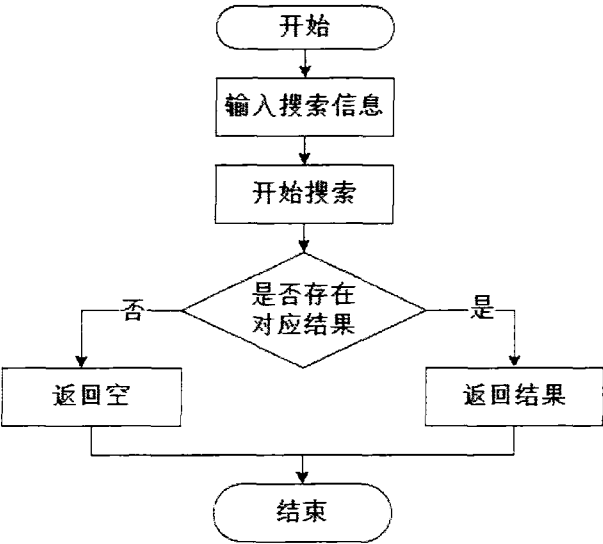


图 4-6 恶意软件检测系统历史记录搜索模块

搜索返回的信息包括软件名称、检测时间和各个模型的检测结果，具体信息如表 4-4 所示：

表 4-4 历史记录搜索返回信息表

序号	字段名	描述
1	文件名称	文件名称
2	检测时间	软件检测时间
3	模型一	恶意软件检测模型：Mal-CLAM 模型
4	模型二	恶意软件检测模型：来自 Xing 等人的研究
5	模型三	恶意软件检测模型：来自 Agrawal 等人的研究
6	模型四	恶意软件检测模型：来自 Catak 等人的研究
7	模型五	恶意软件检测模型：来自 Zhang 等人的研究

4.2.5 可视化模块实现

可视化模块运行流程如图 4-7 所示，该模块提供用户查看近期检测结果的功能，每个页面提供十条检测记录，按照检测时间排序，提供了分页功能，用户可以调整页面查询想要的信息，系统默认初始页码为 1。

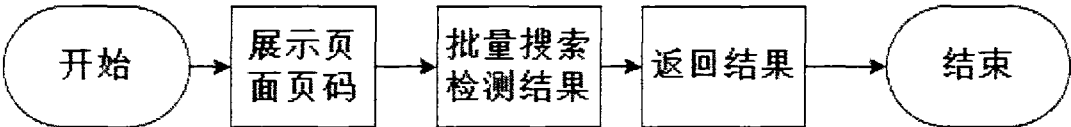


图 4-7 恶意软件检测系统可视化模块

展示的检测信息包括软件名称、检测时间、文件 sha256 值和各个模型的检测结果，具体信息如表 4-5 所示：

表 4-5 检测结果信息表

序号	字段名	描述
1	文件名称	文件名称
2	检测时间	软件检测时间
3	sha256	文件的 sha256 哈希值
4	模型一	恶意软件检测模型：Mal-CLAM 模型
5	模型二	恶意软件检测模型：来自 Xing 等人的研究
6	模型三	恶意软件检测模型：来自 Agrawal 等人的研究
7	模型四	恶意软件检测模型：来自 Catak 等人的研究
8	模型五	恶意软件检测模型：来自 Zhang 等人的研究

4.3 系统测试

本节对系统核心功能进行的测试进行了详细描述，并对系统的使用界面进行了展示。测试使用的软件样本来自于网络，使用了多个从官网上下载的良性软件和从 VirusShare 网站上下载的恶意软件，以便对比系统给出的检测结果是否正确。

(1) 注册功能测试

用户注册模块是该系统的重要组成部分，未经注册并登录的用户没有权限使用该系统的功能，用户注册时不能与已注册用户的用户名重复、密码与确认密码必须相同。

检测系统注册页面如图 4-8 所示。用户输入用户名、密码和确认密码，如果注册失败则会弹窗展示原因说明。

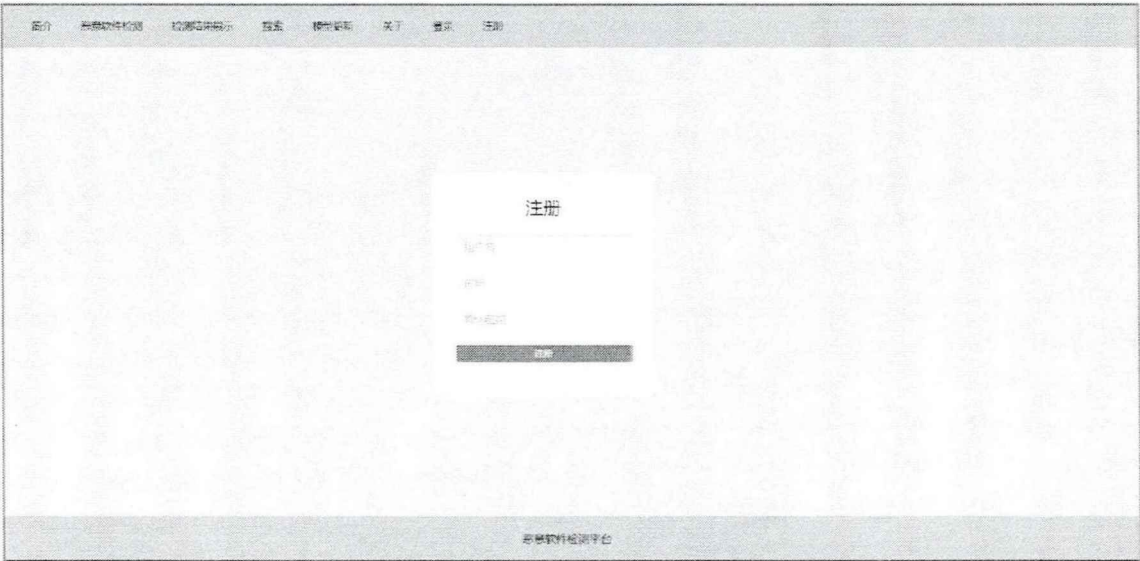


图 4-8 恶意软件检测系统注册界面

系统的注册功能测试结果如表 4-6 所示，通过充分测试不同情形，查看注册

功能是否符合预期设定。

表 4-6 注册功能测试及结果

测试用例编号	用户名	密码	确认密码	预期结果	实际结果	原因说明
1	happyfire	123456789	123456789	注册成功	注册成功	
2	happyfire	123456789	123456789	注册失败	注册失败	用户已注册
3	happy	123456789	123456	注册失败	注册失败	密码不一致
4	happy	123456	123456	注册成功	注册成功	

(2) 登录功能测试

未登录的用户不能使用系统的功能，用户试图使用系统功能时会提示用户登录。用户登录时输入注册好的用户名和密码即可。

检测系统登录页面如图 4-9 所示。用户输入用户名、密码，如果登录失败则会弹窗展示原因说明。



图 4-9 恶意软件检测系统登录界面

系统的登录功能测试结果如表 4-7 所示，通过充分测试不同情形，查看登录功能是否符合预期设定。

表 4-7 登录功能测试及结果

测试用例编号	用户名	密码	预期结果	实际结果	原因说明
1	happyfire	123456789	登录成功	登录成功	
2	happyfire	000000000	登录失败	登录失败	密码错误
3	happyyyy	123456789	登录失败	登录失败	用户不存在
4	happy	123456789	登录成功	登录成功	

(3) 检测功能测试

系统使用了五个模型进行检测，最终结果由其统计结果得出，如果四个及以上模型认定为恶意软件，说明其是高危软件，不建议用户使用；如果二至三个模型认定为恶意软件，说明其是中风险软件，用户谨慎使用；如果零至一个模型认定为恶意软件，说明其是低风险软件，用户可以使用。最后统计多次测试给出的结果，并与测试软件原本属性进行对比，测试系统检测结果是否可靠。

系统恶意软件检测功能测试流程的上传界面和结果展示界面如图 4-10 和图 4-11 所示:

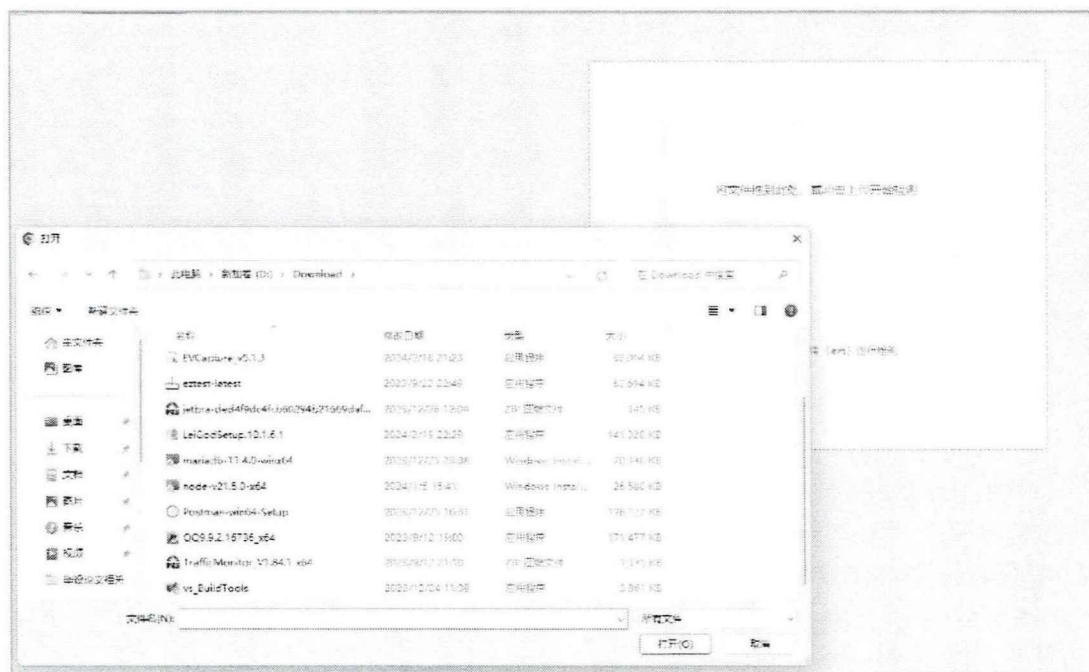


图 4-10 恶意软件检测系统上传界面



图 4-11 恶意软件检测系统检测及结果界面

在检测页面用户点击上传文件或拖拽上传待检测文件后，如图 4-10 所示，系统会将软件样本传输到后台运行检测，并接收后端传回来的检测结果。结果展

示如图 4-11 所示，展示出了软件的名称、散列值特征等，方便用户后续使用查询功能以及其它恶意软件检测模型检测结果等。为了方便用户直观识别出来软件的而已与否，在模型检测结果显示上，如果检测结果为恶意软件则字体显示红色，如果检测结果为良性软件则字体显示为绿色；result 字段是综合了各个模型检测结果给出的风险等级，低风险软件则字体显示为绿色，中风险则字体显示为橙色，高风险则字体显示为红色。这样用户可以只管明确的了解测试软件的性质，从而避免被恶意软件攻击。

系统的检测功能测试结果如表 4-8 所示，通过分别输入良性样本和恶意样本测试，查看系统运行过程是否符合预期设定，检测结果是否正确。

表 4-8 检测功能测试及结果

测试用例名称	恶意软件检测功能
测试流程	1、点击导航栏“恶意软件检测”菜单选项，进入检测页面； 2、按照提示点击“点击上传”链接会弹出文件资源管理窗口； 3、在文件资源管理窗口选取待检测的软件样本上传即可； 4、等待系统返回检测结果。
预期测试结果	软件样本能被系统正常接收； 系统对用户上传的软件进行检测，并能够跳转到结果展示页面； 结果展示页面提供软件的基本信息和各个模型的检测结果。
实际测试结果	系统正常接收软件样本； 系统对用户上传的软件进行检测，并跳转到了结果展示页面； 结果展示页面提供了软件的基本信息和各个模型的检测结果。
结论	检测功能测试通过

测试用例及对应结果如表 4-9 所示，通过充分测试不同样例，查看恶意软件检测功能是否符合预期设定。

表 4-9 恶意软件检测功能测试及结果

测试用例编号	被检测软件	预期结果	实际结果	原因说明
1	vs_BuildTools	良性软件	无风险	vs 构建工具
2	EVCapture_v5.1.3	良性软件	无风险	EV 录屏软件
3	028dfc2431ed56ee063b...	恶意软件	高风险	经处理的恶意软件
4	CF 透视追踪-神龙	恶意软件	高风险	CF 外挂

(4) 模型更新功能测试

模型更新界面如图 4-12 所示。在检测时从后端主动触发系统模型更新功能。经过多次测试系统能够完成模型的更新，每次耗时在 7 小时左右。



图 4-12 恶意软件检测系统模型更新界面

(5) 历史记录搜索功能测试

历史记录搜索功能测试结果如表 4-10 所示，使用软件名称、sha1 值、sha256 值、md5 值等搜索，检测是否能返回对应的查询结果，结果是否正确，是否一致。

表 4-10 历史记录搜索功能测试及结果

测试用例名称	历史记录检测结果搜索功能
测试流程	1、点击导航栏“搜索”菜单选项，进入搜索页面； 2、按照提示输入名称/sha1 值/sha256 值/md5 值，点击进行搜索； 3、等待系统返回搜索信息。
预期测试结果	如果存在对应记录，系统正常返回检测信息； 如果不存在对应记录，系统返回空信息。
实际测试结果	系统正常返回检测信息。
结论	检测记录搜索功能测试通过

检测结果历史记录搜索界面如图 4-13 所示：

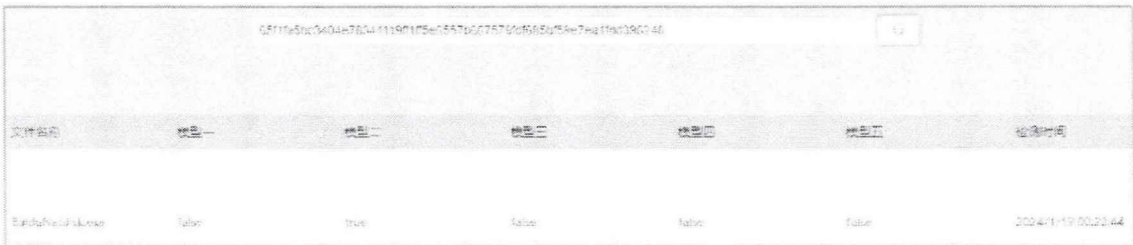


图 4-13 恶意软件检测系统历史记录搜索界面

历史记录搜索界面为用户提供了搜索功能。由于进行一次检测要耗费一定的时间和服务器资源，因此预先搜索软件的检测可以节省许多资源；而且许多用户

缺少安全上网经验，如果不小心将恶意软件下载到自己的计算机上再检测，很可能早已经被其攻击了。用户可以依照软件名称进行检测，不过由于许多恶意软件会取一些迷惑性的名称，因此可能无法得到正确的结果，所以还提过了依靠软件散列值，如 md5 值、sha256 值等，进行检测的功能。

测试用例及对应结果如表 4-11 所示，通过充分测试不同样例，查看历史记录搜索功能是否符合预期设定。

表 4-11 历史记录搜索功能测试及结果

测试用例 编号	关键词	类型	预期结果	实际结果
1	vs_BuildTools	名称	返回检测记录	返回检测记录
2	d96df1051e62aa40baefd512 35be45f8038745582a5d342 8b63123fd2ced60db	sha1	返回检测记录	返回检测记录
3	028dfc2431ed56ee063b98fe 1914c3c0013889010249f6a eb73d8d6fdc5d3801	sha256	返回检测记录	返回检测记录
4	0cef530049b359f56679837e 4bbce04b	md5	无	该软件无检测记录

(6) 可视化功能测试

可视化界面如图 4-14 所示：

文件名称	模型一	模型二	模型三	模型四	模型五	检测时间	md5
GoldfishDisk.exe	false	true	false	false	false	2024/1/19 00:22:44	65f1fa50c2404e70341115ef1f5a6557b607579ed9050f5ee70a10a139c246
TrafficMonitor.exe	false	false	false	false	false	2024/1/19 01:16:43	070911454e9305c9e510b7180fa1adfb3e75570a7428b42c779335753330
jetrun-de489d4f1c080134d21609daf x905305713.exe	false	false	false	false	false	2024/1/19 00:12:21	39de7a0906f19eadc03125e5e20ce7e0e012f6053c70b7215e7be60504409c
图形工具箱2.0.0.0.exe	false	true	false	false	false	2024/1/19 00:06:09	0e0e0e138838ed7ac0b2c4d61791c990e5365d27e01e05f0c05054d0f061a
图形工具箱2.0.0.0.exe	false	true	false	false	false	2024/1/19 00:04:29	c0b9ec0050584c40e013d290e5b7e5cf3473a170fb44199ac8016e10f0d0742
TrafficMonitor_v1.54.1_x64.exe	false	true	false	false	false	2024/1/19 01:03:11	7217c296d50936c56a36f8a90f0a50537ae075d570c38c54d46393a6f0d702
0x0405C4090e1a2e03390903c7e4 d1c06a6305c0ac96c1350769dc53c2 .exe	true	true	false	true	true	2024/1/18 23:59:01	ead170d309d403cdad5c09f9a02a29e21770b0e05e222ce00eb5afac80f0f
0x0405C4090e1a2e03390903c7e4 d2c0906105c0ac96c1350769dc53c2 1.exe	true	true	false	true	true	2024/1/18 23:55:10	ead170d309d403cdad5c09f9a02a29e21770b0e05e222ce00eb5afac80f0f

图 4-14 恶意软件检测系统可视化界面

可视化界面用户可以看到系统近期所作的所有软件的检测记录，可以看到各个模型对该软件的详细检测结果和软件信息；由于检测记录众多，本系统还做了分页处理，按照检测时间排序，最新检测的软件信息排在前面。

可视化功能测试结果如表 4-12 所示，测试相应操作能否顺利执行。

表 4-12 可视化功能测试及结果

测试用例名称	可视化功能
测试流程	1、点击导航栏“检测结果展示”菜单选项，进入对应页面； 2、记录页面展示最近十次检测的结果； 3、记录页面设置了分页，点击页码跳转到指定记录页面。
预期测试结果	系统能够正常跳转到记录页面，展示检测记录； 系统根据用户的点击，能够跳转到指定页面，展示检测记录。
实际测试结果	系统正常跳转到记录页面，展示了检测记录； 系统根据用户点击，正确跳转到了指定页面，展示了检测记录。
结论	可视化功能测试通过

4.4 本章小结

本章分析了现有恶意软件检测检测系统的功能需求，对系统的架构进行设计，并基于本文研究成果实现了恶意软件检测系统。系统包括登录注册、恶意软件检测、模型更新、历史记录搜索和可视化五个主要功能；本文还设计了全面的测试案例，对系统的核心功能进行了测试和验证。系统使用简单，检测结果准确，返回信息全面详细。

第五章 总结与展望

随着互联网的普及和计算机技术的发展,网络攻击事件越来越多,安全形势越来越严峻,吸引了越来越多的人关注和研究。面对恶意软件种类的不断出新和攻击次数的爆炸式增长,快速准确的识别恶意软件是维护网络空间安全的一个重要研究方向。深度学习作为当前人工智能领域的研究热点,已经在文本处理、图像音频处理等方面取得了令人瞩目的研究成果,因此可以将深度学习用于恶意软件检测,实现检测的更加精准和更加可靠。本文提出了一种基于 API 调用序列深层特征和依赖关系的深度学习恶意软件检测模型 Mal-CLAM,从不同角度提取 AP 序列的特征,然后使用机器学习和深度学习算法模型训练,通过结构优化和参数调整提升模型的检测性能,并开发了恶意软件检测系统。

5.1 研究工作总结

API 调用序列是恶意软件检测动态分析中最主要的特征,本文通过对相关检测方法的研究,总结当前基于 API 序列特征进行恶意软件检测方法的不足,优化特征提取和检测模型,针对 Windows 系统上的可执行文件进行检测,实现了基于 API 序列特征和依赖关系的深度学习恶意软件检测模型 Mal-CLAM 模型,能够有效检测恶意软件。本文的主要研究成果如下:

(1) 本文研究并实现了 Mal-CLAM 模型,结合 API 序列结构特征和上下文依赖关系进行恶意软件检测。首先,将 API 序列去重处理,并统一设置每个序列包含 1000 个 API,然后使用多个 Gated-CNN 捕获 API 序列的深度特征,并利用 Bi-LSTM 提取序列之间的上下文依赖关系;使用深度自编码器捕获序列深层特征;最后将提取的特征融合,使用 MLP 模型检测。

Mal-CLAM 模型减少了人为因素的参与,加速了模型更新,达到了 94.49% 的准确率、92.75% 的召回率、92.63% 的 F1 值和 98.43% 的 AUC 值。本文通过测试了多个结构的消融实验证明了所选模型的优越性;本文还通过在实验中将干扰样本引入训练数据集的方法,证明了 Mal-CLAM 模型相比其它检测模型具有更强的抗干扰能力和鲁棒性。

(2) 本文基于所研究的检测模型建立一个恶意软件检测平台,整合了本文的研究结果,将其放入实际应用中,为对抗恶意软件做出积极有效的贡献。通过需求分析、架构设计明确,系统的主要提供功能和组织框架。系统整体采用前后端分离架构,前端采用 Vue 框架编写,展示在浏览器上能够兼容 Microsoft Edge、Google Chrome 等主流浏览器;后端使用 Python 编程语言、Fastapi 框架编写;数

数据库使用 MySQL 编写。检测系统主要包括登录注册、恶意软件检测、模型更新、历史记录搜索、可视化等功能模块，便于用户了解安全态势、保护计算机安全，避免被恶意软件攻击。

5.2 未来工作展望

本文设计并实现了基于 API 序列的深度学习恶意软件检测模型，该方法在检测上虽然取得了不错的成绩，但是仍有很大的提升空间。本文的最终研究目标是创建一个准确率更高、鲁棒性更强、泛用性更广的检测模型，可以有效地处理各种类型的恶意软件。具体的深入研究方向如下：

(1) 本文以 API 调用序列为软件特征，这需要在安全虚拟环境中运行获取。但是随着反虚拟化技术的发展恶意软件在虚拟环境中会逃避检测不执行恶意行为，或者执行中插入大量正常的 API 调用，稀释获取的 API 序列包含的信息。因此考虑结合软件的静态特征，采用动静结合分析的方法进行检测。

(2) 本文采用 Cuckoo2 沙箱作为安全虚拟环境获取 API 序列，但是其被大量研究使用，当前许多恶意软件都会针对 Cuckoo2 沙箱进行规避操作，导致难以获取有信息。针对这个问题，后续可以考虑使用一些新型沙箱，如 Hybrid Analysis 沙箱、Any.Run 云沙箱等，这些新型沙箱既可以提取恶意软件运行信息，又可以根据需求定制化功能，并且通过云上服务减轻本地资源压力，提高系统检测效率。

(3) 本文采用的 API 序列是去重后的长度为 1000 的序列，后续可以研究不同序列长度对检测性能的影响；当前 API 的选取是基于时间顺序的，后续可以考虑采用出现频率、TF-IDF 选取更有价值的 API；或者以多个 API 函数组成的调用组合为单位进行选取。

(4) 本文采用的深度学习模型是一种黑盒模型，虽然可以得到较好的检测性能，但对于其实现原理并不明晰。下一步计划更深入地研究模型中每个组件的具体作用，进一步增强模型的可解释性。当前已经有研究触及到这方面，如特征重要性分析、局部解释方法、特征可视化等研究方法。理解深度学习的运行原理对提高模型检测性能具有十分重要的作用。

参考文献

- [1] Amoroso E. Recent progress in software security[J]. IEEE Software, 2018, 35(2): 11-13.
- [2] 2023 AV-TEST [EB/OL]. <https://www.av-test.org>.
- [3] Ghafur S, Kristensen S, Honeyford K, et al. A retrospective impact analysis of the WannaCry cyberattack on the NHS[J]. NPJ digital medicine, 2019, 2(1): 98.
- [4] Benmalek M. Ransomware on cyber-physical systems: Taxonomies, case studies, security gaps, and open challenges[J]. Internet of Things and Cyber-Physical Systems, 2024.
- [5] Willett M. Lessons of the SolarWinds hack[M]//Survival April–May 2021: Facing Russia. Routledge, 2023: 7-25.
- [6] Tsvetanov T, Slaria S. The effect of the Colonial Pipeline shutdown on gasoline prices[J]. Economics Letters, 2021, 209: 110122.
- [7] Canfora G, Mercaldo F, Visaggio C A, et al. Metamorphic malware detection using code metrics[J]. Information Security Journal: A Global Perspective, 2014, 23(3): 57-67.
- [8] Cortial K, Pachot A. Sodinokibi intrusion detection based on logs clustering and random forest[C]//2021 2nd International Conference on Artificial Intelligence and Information Systems. 2021: 1-4.
- [9] O'Kane P, Sezer S, McLaughlin K. Obfuscation: The hidden malware[J]. IEEE Security & Privacy, 2011, 9(5): 41-47.
- [10] Corlatescu D G, Dinu A, Gaman M P, et al. EMBERSim: A Large-Scale Databank for Boosting Similarity Search in Malware Analysis[J]. Advances in Neural Information Processing Systems, 2024, 36.
- [11] 申培, 刘福龙, 桑海伟. 恶意代码检测研究综述[J]. 重庆理工大学学报 (自然科学), 2022, 36(11): 212-218.
- [12] El Merabet H, Hajraoui A. A survey of malware detection techniques based on machine learning[J]. International Journal of Advanced Computer Science and Applications, 2019, 10(1).
- [13] Aslan Ö A, Samet R. A comprehensive review on malware detection approaches[J]. IEEE access, 2020, 8: 6249-6271.
- [14] Zolkipli M F, Jantan A. A framework for malware detection using combination technique and signature generation[C]//2010 Second International Conference on

- Computer Research and Development. IEEE, 2010: 196-199.
- [15] Borojerdi H R, Abadi M. MalHunter: Automatic generation of multiple behavioral signatures for polymorphic malware detection[C]//ICCKE 2013. IEEE, 2013: 430-436.
- [16] 梅瑞, 严寒冰, 沈元, 等. 二进制代码切片技术在恶意代码检测中的应用研究[J]. Journal of Cyber Security 信息安全学报, 2021, 6(3).
- [17] Ye Y, Li T, Jiang Q, et al. CIMDS: adapting postprocessing techniques of associative classification for malware detection[J]. IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), 2010, 40(3): 298-307.
- [18] Arnold W, Tesauo G. Automatically generated Win32 heuristic virus detection[C]//Proceedings of the 2000 international virus bulletin conference. 2000.
- [19] 张吉昕. 基于深度学习的恶意软件特征分析与检测方法研究[D]. 湖南大学, 2019.
- [20] Wolsey A. The State-of-the-Art in AI-Based Malware Detection Techniques: A Review[J]. arXiv preprint arXiv:2210.11239, 2022.
- [21] Liu Y, Tantithamthavorn C, Li L, et al. Deep learning for android malware defenses: a systematic literature review[J]. ACM Computing Surveys, 2022, 55(8): 1-36.
- [22] Peiravian N, Zhu X. Machine learning for android malware detection using permission and api calls[C]//2013 IEEE 25th international conference on tools with artificial intelligence. IEEE, 2013: 300-305.
- [23] 崔晓龙, 简川杰, 刘欣, 等. 基于多安全机制的轻量级 Linux 沙箱设计与实现[J]. 实验室研究与探索, 2023, 42(9): 83-87.
- [24] Xiaofeng L, Fangshuo J, Xiao Z, et al. ASSCA: API sequence and statistics features combined architecture for malware detection[J]. Computer Networks, 2019, 157: 99-111.
- [25] Das S, Liu Y, Zhang W, et al. Semantics-based online malware detection: Towards efficient real-time protection against malware[J]. IEEE transactions on information forensics and security, 2015, 11(2): 289-302.
- [26] Cai H. Assessing and improving malware detection sustainability through app evolution studies[J]. ACM Transactions on Software Engineering and Methodology (TOSEM), 2020, 29(2): 1-28.
- [27] 姚烨, 钱亮, 朱怡安, 等. 一种基于混合特征的移动终端恶意软件检测方法[J]. 信息安全学报, 2022, 7(2): 120-138.
- [28] Saracino A, Sgandurra D, Dini G, et al. Madam: Effective and efficient behavior-

- based android malware detection and prevention[J]. IEEE Transactions on Dependable and Secure Computing, 2016, 15(1): 83-97.
- [29] Wen L, Yu H. An Android malware detection system based on machine learning[C]//AIP conference proceedings. AIP Publishing, 2017, 1864(1).
- [30] Li J, Wang Z, Wang T, et al. An android malware detection system based on feature fusion[J]. Chinese Journal of Electronics, 2018, 27(6): 1206-1213.
- [31] Shaukat K, Luo S, Varadharajan V, et al. A survey on machine learning techniques for cyber security in the last decade[J]. IEEE access, 2020, 8: 222310-222354.
- [32] Dong S, Wang P, Abbas K. A survey on deep learning and its applications[J]. Computer Science Review, 2021, 40: 100379.
- [33] Kamilaris A, Prenafeta-Boldú F X. Deep learning in agriculture: A survey[J]. Computers and electronics in agriculture, 2018, 147: 70-90.
- [34] Guo Y, Liu Y, Oerlemans A, et al. Deep learning for visual understanding: A review[J]. Neurocomputing, 2016, 187: 27-48.
- [35] 祝士祥. 基于深度学习的工控网络异常检测研究和实现[D]. 北京邮电大学, 2017.
- [36] 严春满, 王铖. 卷积神经网络模型发展及应用[J]. Journal of Frontiers of Computer Science & Technology, 2021, 15(1).
- [37] Jha S, Prashar D, Long H V, et al. Recurrent neural network for detecting malware[J]. computers & security, 2020, 99: 102037.
- [38] 景鸿理, 黄娜, 李建国. 基于机器学习的恶意软件检测研究进展及挑战[J]. 信息技术与网络安全, 2020, 39(11): 38-44.
- [39] Naeem H, Guo B, Naeem M R. A light-weight malware static visual analysis for IoT infrastructure[C]//2018 International conference on artificial intelligence and big data (ICAIBD). IEEE, 2018: 240-244.
- [40] Xing X, Jin X, Elahi H, et al. A malware detection approach using autoencoder in deep learning[J]. IEEE Access, 2022, 10: 25696-25706.
- [41] Hassen M, Carvalho M M, Chan P K. Malware classification using static analysis based features[C]//2017 IEEE Symposium Series on Computational Intelligence (SSCI). IEEE, 2017: 1-7.
- [42] Alzaylaee M K, Yerima S Y, Sezer S. DL-Droid: Deep learning based android malware detection using real devices[J]. Computers & Security, 2020, 89: 101663.
- [43] Raff E, Barker J, Sylvester J, et al. Malware detection by eating a whole exe[C]//Workshops at the thirty-second AAAI conference on artificial intelligence.

2018.

- [44] Cui L, Cui J, Ji Y, et al. API2Vec: Learning Representations of API Sequences for Malware Detection[C]//Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. 2023: 261-273.
- [45] Zou D, Wu Y, Yang S, et al. IntDroid: Android malware detection based on API intimacy analysis[J]. ACM Transactions on Software Engineering and Methodology (TOSEM), 2021, 30(3): 1-32.
- [46] Ma Z, Ge H, Liu Y, et al. A combination method for android malware detection based on control flow graphs and machine learning algorithms[J]. IEEE access, 2019, 7: 21235-21245.
- [47] Agrawal R, Stokes J W, Marinescu M, et al. Neural sequential malware detection with parameters[C]//2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2018: 2656-2660.
- [48] Saad S, Briguglio W, Elmiligi H. The curious case of machine learning in malware detection[J]. arXiv preprint arXiv:1905.07573, 2019.
- [49] 王天歌. 基于 API 调用序列的 Windows 平台恶意代码检测方法[D]. 北京交通大学, 2021.
- [50] Upadhyay R K, Singh P. Modeling and control of computer virus attack on a targeted network[J]. Physica A: Statistical Mechanics and its Applications, 2020, 538: 122617.
- [51] 王瑞玲, 薛亚奎. 具有饱和发生率的网络蠕虫病毒模型分析[J]. 华中师范大学学报 (自然科学版), 2023, 57(3): 347-353.
- [52] 胡建伟, 张玉, 崔艳鹏. Android 勒索软件防护技术研究[J]. 计算机工程与科学, 2020, 42(4): 610-619.
- [53] 郑文庚, 李凌崴, 廖广军. Windows 系统环境下基于内存分析的木马病毒取证[J]. Forensic Science & Technology, 2020, 45(6).
- [54] 谭清尹, 曾颖明, 韩叶, 等. 神经网络后门攻击研究[J]. 网络与信息安全学报, 2021, 7(3): 46-58.
- [55] 李佳琳, 王雅哲, 罗吕根, 等. 面向安卓恶意软件检测的对抗攻击技术综述[J]. 信息安全学报, 2021, 6(4): 28-43.
- [56] Taheri L, Kadir A F A, Lashkari A H. Extensible android malware detection and family classification using network-flows and API-calls[C]//2019 International Carnahan Conference on Security Technology (ICCST). IEEE, 2019: 1-8.
- [57] Chen X, Hao Z, Li L, et al. Cruparamer: Learning on parameter-augmented api sequences for malware detection[J]. IEEE Transactions on Information Forensics and

- Security, 2022, 17: 788-803.
- [58] Jalilian A, Narimani Z, Ansari E. Static signature-based malware detection using opcode and binary information[C]//Data Science: From Research to Application. Springer International Publishing, 2020: 24-35.
- [59] Fan M, Liu J, Luo X, et al. Android malware familial classification and representative sample selection via frequent subgraph analysis[J]. IEEE Transactions on Information Forensics and Security, 2018, 13(8): 1890-1905.
- [60] DeRose J F, Wang J, Berger M. Attention flows: Analyzing and comparing attention mechanisms in language models[J]. IEEE Transactions on Visualization and Computer Graphics, 2020, 27(2): 1160-1170.
- [61] Zhang Z, Wang H, Xu F, et al. Complex-valued convolutional neural network and its application in polarimetric SAR image classification[J]. IEEE Transactions on Geoscience and Remote Sensing, 2017, 55(12): 7177-7188.
- [62] Jung J, Kim H, Shin D, et al. Android malware detection based on useful API calls and machine learning[C]//2018 IEEE First International Conference on Artificial Intelligence and Knowledge Engineering (AIKE). IEEE, 2018: 175-178.
- [63] Zhang Z, Qi P, Wang W. Dynamic malware analysis with feature engineering and feature learning[C]//Proceedings of the AAAI conference on artificial intelligence. 2020, 34(01): 1210-1217.
- [64] Catak F O, Yazı A F, Elezaj O, et al. Deep learning based Sequential model for malware analysis using Windows exe API Calls[J]. PeerJ Computer Science, 2020, 6: e285.
- [65] Alzubaidi L, Zhang J, Humaidi A J, et al. Review of deep learning: concepts, CNN architectures, challenges, applications, future directions[J]. Journal of big Data, 2021, 8: 1-74.
- [66] 鄢然、廖记登、吴小勇, 等. 基于卷积神经网络的砂石骨料分类方法研究[J]. Laser & Optoelectronics Progress, 2021, 58(20): 2010015.
- [67] 王欣雅, 林泓旭, 吕尚颖, 等. 基于深度学习的佩戴口罩下的人脸识别[J]. Computer Science and Application, 2023, 13: 1576.
- [68] Amer E, Zelinka I, El-Sappagh S. A multi-perspective malware detection approach through behavioral fusion of api call sequence[J]. Computers & Security, 2021, 110: 102449.
- [69] DiPietro R, Hager G D. Deep learning: RNNs and LSTM[M]//Handbook of medical image computing and computer assisted intervention. Academic Press, 2020: 503-

519.

- [70] Xiao X, Zhang S, Mercaldo F, et al. Android malware detection based on system call sequences and LSTM[J]. Multimedia Tools and Applications, 2019, 78: 3979-3999.
- [71] Mathew J, Ajay Kumara M A. API call based malware detection approach using recurrent neural network—LSTM[C]//Intelligent Systems Design and Applications: 18th International Conference on Intelligent Systems Design and Applications (ISDA 2018) held in Vellore, India, December 6-8, 2018, Volume 1. Springer International Publishing, 2020: 87-99.
- [72] Pinaya W H L, Vieira S, Garcia-Dias R, et al. Autoencoders[M]//Machine learning. Academic Press, 2020: 193-208.
- [73] Wang W, Zhao M, Wang J. Effective android malware detection with a hybrid model based on deep autoencoder and convolutional neural network[J]. Journal of Ambient Intelligence and Humanized Computing, 2019, 10: 3035-3043.
- [74] Purnama B, Stiawan D, Hanapi D, et al. N-gram effect in malware detection using multilayer perceptron (mlp)[C]//2021 8th International Conference on Electrical Engineering, Computer Science and Informatics (EECSI). IEEE, 2021: 45-49.
- [75] Zhang S, Wu J, Zhang M, et al. Dynamic Malware Analysis Based on API Sequence Semantic Fusion[J]. Applied Sciences, 2023, 13(11): 6526.
- [76] Dauphin Y N, Fan A, Auli M, et al. Language modeling with gated convolutional networks[C]//International conference on machine learning. PMLR, 2017: 933-941.
- [77] Ioffe S, Szegedy C. Batch normalization: Accelerating deep network training by reducing internal covariate shift[C]//International conference on machine learning. pmlr, 2015: 448-456.
- [78] Jeon J, Jeong B, Baek S, et al. Hybrid malware detection based on bi-lstm and spp-net for smart iot[J]. IEEE Transactions on Industrial Informatics, 2021, 18(7): 4830-4837.
- [79] Fenanir S, Semchedine F, Harous S, et al. A Semi-supervised Deep Auto-encoder Based Intrusion Detection for IoT[J]. Ingénierie des Systèmes d'Information, 2020, 25(5).
- [80] Wei C, Kakade S, Ma T. The implicit and explicit regularization effects of dropout[C]//International conference on machine learning. PMLR, 2020: 10181-10192.
- [81] Greamo C, Ghosh A. Sandboxing and virtualization: Modern tools for combating malware[J]. IEEE Security & Privacy, 2011, 9(2): 79-82.

- [82]于莹, 杨婷婷, 杨博雄. 混淆矩阵分类性能评价及 Python 实现[J]. 现代计算机, 2021, 20: 70-73.